Capability Capsule in AI4Research

Muk Chunpongtong · September 2026

This report covers 4 subjects, in the same order as the presentation:

- What Capability Capsule is for, and the platform it builds on
- What prior work shows, each result followed by the design rule it leads to
- What already exists in AI4Research
- How Capability Capsule should be designed, evaluated and built

Claims about AI4Research were checked in its source code at one pinned commit, and claims about DeepSeek Harness (DSH) were checked against its documentation at one pinned commit. DSH has not been run. This version of the report keeps code paths and line numbers out of the text; the annotated variant, kept beside it, carries the location behind every code claim.

Appendix A holds the paper details, Appendix B the AI4Research inventory, Appendix C the manifest fields and rejection reasons, Appendices D and E the DSH and MCP facts, Appendix F the task-graph dispatch rules, and Appendix G the design details that the presentation leaves out.

1. The goal: agents that build, check and keep their own capabilities

Capability Capsule is the part of AI4Research that declares, admits and selects the capabilities agents use. Today every capsule is written by hand, and none runs on DSH. The goal of this project is a system that improves itself and its tools by keeping a library of capabilities it adds to while it runs.

AI4Research itself is a local-first multi-agent framework. It compiles a natural-language goal into a governed task graph and runs that graph across swappable AI, CLI and tool operators. Capability Capsule is to move onto DSH and be extended so that agents create, verify, update and use capabilities themselves.

The need comes from run length. As agent loops reach days and weeks, agents need more abilities, tools, skills and MCP servers than humans can hand-write, hand-verify or predict. Akin to how people learn new skills and become more proficient with experience, agents need to learn and develop themselves, reacting to their needs and improving over time, while keeping extra cost low, favouring reliability and recovery, and preventing catastrophic failure.

This means agents must be capable of:

- Exploring the capability space
- Deciding whether to use any skills
- Choosing a skill to use and deciding if a new skill should be made
- Deciding whether a skill should be modified or retired
- Implementing new skills
- Evaluating the functionality of a new skill
- Managing the capability space of existing skills when new ones are created
- Handling skill failure to determine downstream impacts and fix errors
- Maintaining provenance for observability and tracing

Section 4 addresses each of these: the generalist capsule and selection in Sections 4.1 and 4.4, the decision to build in Section 4.2, implementation and evaluation in Section 4.3, modifying, retiring and managing the library in Section 4.5, handling failure in Sections 4.3 and 4.5, and provenance in the manifest of Section 4.3.

A system like this already exists. Alita-G [2] implements the loop this project sets as its goal and measures it on a fixed benchmark (Section 2.1). Thus, the question this report answers is what has to change for research runs that last days and weeks.

1.1 MCP is a protocol, not a tool

MCP is often read as a synonym for tool. It is a stateless client-server protocol for reaching tools, data and prompts [10]. A host application, such as an agent runtime, runs 1 client per server connection and mediates between them, so a server never sees another server's data. Messages follow JSON-RPC 2.0, and since the 2026-07-28 revision there is no handshake, because every request carries its own protocol version and capabilities.

Capabilities flow in both directions:

- Servers offer tools, which are actions, resources, which are data such as files or database rows, and prompts, which are message templates
- Clients offer sampling, where a server asks the host's model for a completion, elicitation, where a server asks the user for input, and roots, which bound the files a server may use
- Optional extensions add long-running tasks, inline interactive apps and skills delivered over MCP

Thus, a tool is 1 of 3 server primitives, and an MCP server is a route to capabilities rather than a capability (Fig. 1). Appendix E lists the MCP facts this report relies on.

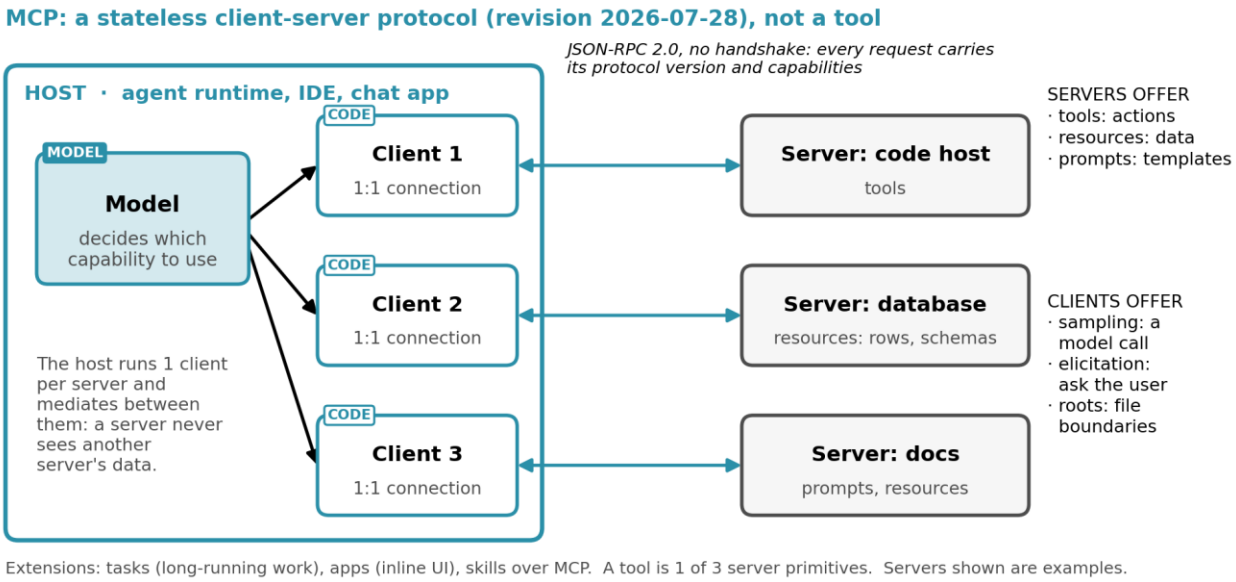


Fig. 1. MCP as a client-server protocol. The host runs 1 client per server; servers offer tools, resources and prompts, and clients offer sampling, elicitation and roots. The example servers are illustrative.

1.2 DeepSeek Harness: every layer is a plugin, and a capsule wraps each of them

DSH is the target platform. It is a plugin harness with no privileged core: the model adapter, tool registry, session log and agent loop are themselves plugins. It is built on Cordis, a meta-framework described in "A programming paradigm for spatiotemporal composability" [1].

4 properties of DSH shape the design:

- Loading and unloading: everything a plugin registers unwinds when it unloads

- Versioned data: committed generations are never renamed, replaced or deleted, and a write publishes a version-named successor beside its source
- Logging: anything that reaches a model request must be reconstructable from the session log
- Sandboxing: sandbox policy is a separate package from the backends for Linux, macOS and Windows

Spatiotemporal composability is the name Cordis gives to the 2 properties those 4 rest on, and both do work in this design [1, Sec. 1.1]. The time half, temporal composability, means that everything a component registers is an effect that carries its own inverse, so a component can be loaded, replaced and withdrawn while the system keeps serving, and everything it added is reclaimed when it goes. The space half, spatial composability, means that components declare and resolve their dependencies, so the effects of distinct components interleave without disturbing one another, and a dependent deactivates when what it depends on is withdrawn [1, Sec. 3, Sec. 3.2].

Cordis states what a self-evolving harness loses without each half. Without the time half, every change to a capability forces a restart that discards in-flight work. Without the space half, a replacement silently breaks the components that depended on it, and the breakage surfaces only at reload time [1, Sec. 1.2.2]. Section 4.5 rests on the time half, Sections 4.1 and 4.2 on the space half, and Appendix D maps both onto DSH.

DSH is a developer preview at version 0.1.5-rc.2, and this report relies on its documented services rather than on a running integration (Appendix D). DSH settles loading, swapping, reverting, sandboxing and logging. It does not settle the contract, meaning what a capability may do and what must be true before it is admitted.

A capsule is that contract. Many agent systems have several layers of abilities, and Capability Capsule aims to merge them into 1 abstraction via DSH, where everything is a plugin (Fig. 2). The merge is a project requirement rather than a conclusion of this report. The 4 layers are:

- Skills: how to reason and perform a workflow
- Tools: individual executable actions
- MCP: how agents connect to external tools
- A2A: how other agents are discovered and delegated to

A capsule answers the question the other layers leave open: whether a capability may run, and what it will do to the system. It is a manifest around any of the 4 layers that declares a typed contract, the capability's effects, a trust class and held-out tests. MCP answers how an agent reaches a tool and A2A answers how an agent delegates to a peer, but neither says where a capability comes from or what must be true before it is trusted. The question matters when humans write capabilities, and it becomes essential once the system starts writing them itself.

The merge has 2 limits in DSH today: DSH supports MCP tools but not MCP resources or prompts, and it names A2A as a planned transport rather than an existing one.

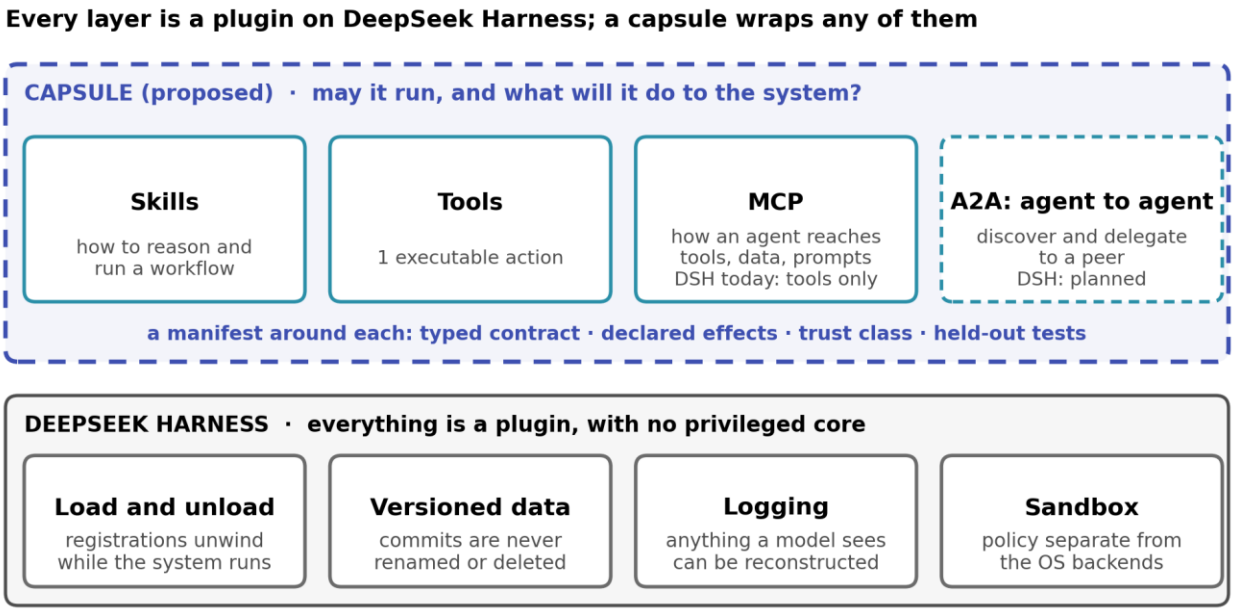


Fig. 2. Every layer is a plugin on DSH, and a capsule wraps any of them. The capsule band is proposed; A2A is dashed because DSH names it as planned.

2. The evidence: each result, and the design rule it leads to

Prior work shows 2 things. An agent can build its own capability library and improve. And judging what it builds only by task outcome fails at the tool, skill and agent levels. Each subsection states a result and its limits, and the design rules R1 to R6 follow the results that lead to them. Appendix A holds the per-paper details.

2.1 Alita-G: a self-growing tool library already works

Alita-G [2] already implements the loop this project sets as its goal, and it measurably improves the agent. A general-purpose master agent writes raw MCP tools while solving specific tasks, generalizes and redocuments them, and places them in a library called the MCP Box. Specialist agents then select MCP tools for their tasks by semantic-similarity search over the library (Fig. 3).

On the 165-question GAIA validation set of real-world tasks, Alita-G reports 83.03% pass@1 with the generated library, relative to 75.15% for the same agent without it. Mean token use also falls by 15.5%, from 12,305 to 10,394 per task [2, Table 1].

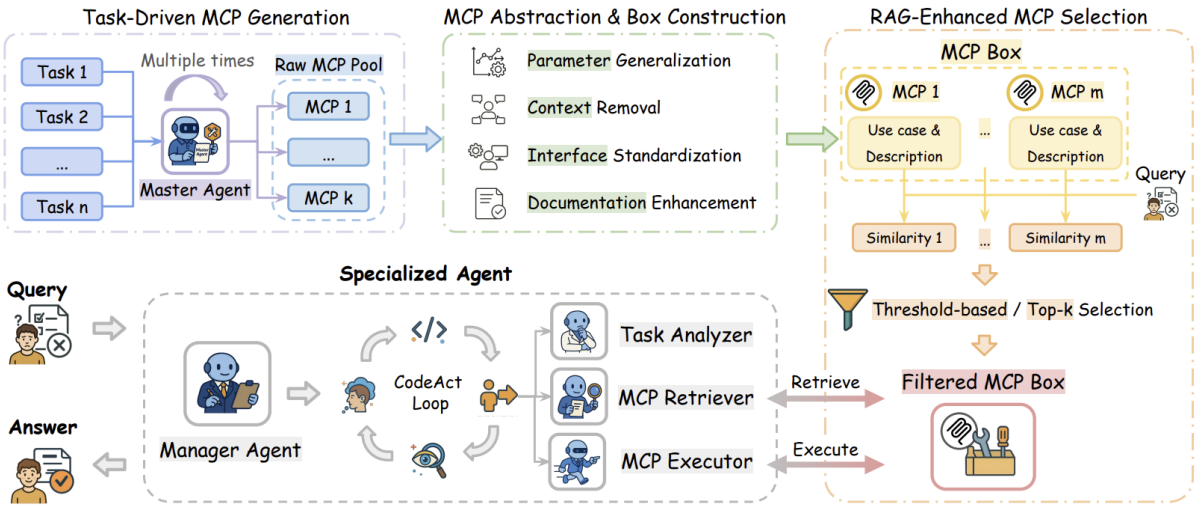


Fig. 3. The Alita-G workflow: task-driven MCP generation, abstraction into the MCP Box, and similarity-based selection by a specialized agent. Reproduced from [2, Fig. 1].

The loop runs in 4 steps [2, Sec. 3]:

- Harvesting, where a master agent solves each task and writes MCP tools for reusable sub-solutions
- Keeping every tool generated in a run whose final answer was correct [2, Eq. (5)]
- Abstracting each kept tool with 1 LLM rewrite that generalizes parameters, removes task context, standardizes the interface to FastMCP and adds documentation [2, Eq. (6)]
- Reusing tools through a specialist that embeds the query, keeps tools whose description and use case exceed a cosine similarity of 0.7, and solves the task in a CodeAct loop

There are three limitations to this result:

- The tool library is harvested from the same GAIA validation set the score is reported on, which bounds the claim to domain specialization rather than held-out generalization
- GAIA totals are not directly comparable between systems, since Alita-G runs Claude Sonnet 4 as manager with GPT-4.1 as web agent, and other systems use different stacks
- The GAIA figures are single runs, since the paper reports no seeds, standard errors or confidence intervals, and the 7.88-point gain is 13 questions turning from wrong to right out of 165 [2, Table 6]

2.2 Where Alita-G falls short, and why a research setting is harder

Alita-G curates its library on 1 signal: whether the final answer was correct [2, Eq. (5)]. The rule keeps every MCP tool generated in a successful run, with no term for whether that tool contributed to the answer, so the system has no per-tool measure of correctness or quality. Table 1 sets each of its choices against what goes wrong and against the setting of AI4Research, where a sprint is 1 request run on 1 frozen plan.

Alita-G	What goes wrong	AI4Research
Keeps every tool from a run with a correct answer [2, Eq. (5)]	Tools that did not help are kept too	Research steps often have no correct answer
Rewrites each kept tool once with an LLM and never tests the result [2, Eq. (6)]	The version kept is untested	Capsules are reused for weeks

Alita-G	What goes wrong	AI4Research
Never removes duplicates, a stated choice made to preserve coverage	As the library grows from 26 to 128 tools, the share of distinct tools (clusters at similarity 0.7 per tool) falls from 100% to 51% and accuracy plateaus [2, Table 4]	The library never stops growing
Picks tools by text similarity above a threshold	Changing the threshold moves accuracy by 16 points on 25 questions [2, Table 3]	Selection by typed contracts checked by code, then a planner model
Builds once for 1 fixed benchmark, then freezes	Tools come from the questions they are scored on	New tasks keep arriving, and the library changes while sprints run

Table 1. Where Alita-G falls short, and why the setting of AI4Research is harder.

Appendix A.1 compares the 2 settings in full and lists the smaller caveats, including 2 answers that flipped from right to wrong.

Thus, the design keeps Alita-G's loop and adopts 4 rules from its limits and its setting:

- R1: capsules are built at the same time as they are used, on a separate clock, so a sprint never waits for a build
- R2: every sprint pins the exact capsule versions it uses, so the library can change while sprints run, and a new capsule is used from the next sprint
- R3: every capsule is validated before it is kept; Sections 2.3 to 2.7 establish what validation must mean
- R4: duplicates are rejected, and a capsule close to an existing one is versioned rather than copied

The same table also fixes what a build budget is for. Alita-G stops harvesting after 3 rounds, because the step from 3 rounds to 4 adds 28 MCP tools and only 8 new clusters while average accuracy stays at 83.03%, and the authors read that as the point where a round buys near-duplicates rather than capabilities [2, Table 4]. Thus, a growing library stops paying before it stops growing, which is why the cold path builds under a budget at all (Appendix G.4), and why Section 5.1 tests whether reuse repays admission instead of assuming it.

2.3 Beyond Task Completion: 96.8% of kept tools fail tests of their own job

Beyond Task Completion [3] asks how well self-made tools work on inputs beyond the task that produced them. The answer is that tools checked only through task outcomes are broken, with nothing in the session flagging it.

The authors preserved the source of every tool that a tool-evolving harness synthesized with no per-tool check. They then replayed each tool against a held-out conformance suite for the capability it was built for, made of hidden unit inputs and adversarial probes. Across 222 tools from 3 tool-creating protocols, run with Claude Haiku 4.5 over 3 seeds by 8 sessions of EvolveTool-Bench each, 215 (96.8%) scored zero correctness [3, Table 5] while executing cleanly and raising no in-session flag (Fig. 4). The sessions carrying these tools still passed 31% to 34% of their verified tasks [3, Sec. 5.3]; Table A4 gives 28.7% to 35.8% for the earlier run of the same matrix. The paper calls this silent rot and traces it to a verifier that "scores task outputs, not tools" [3].

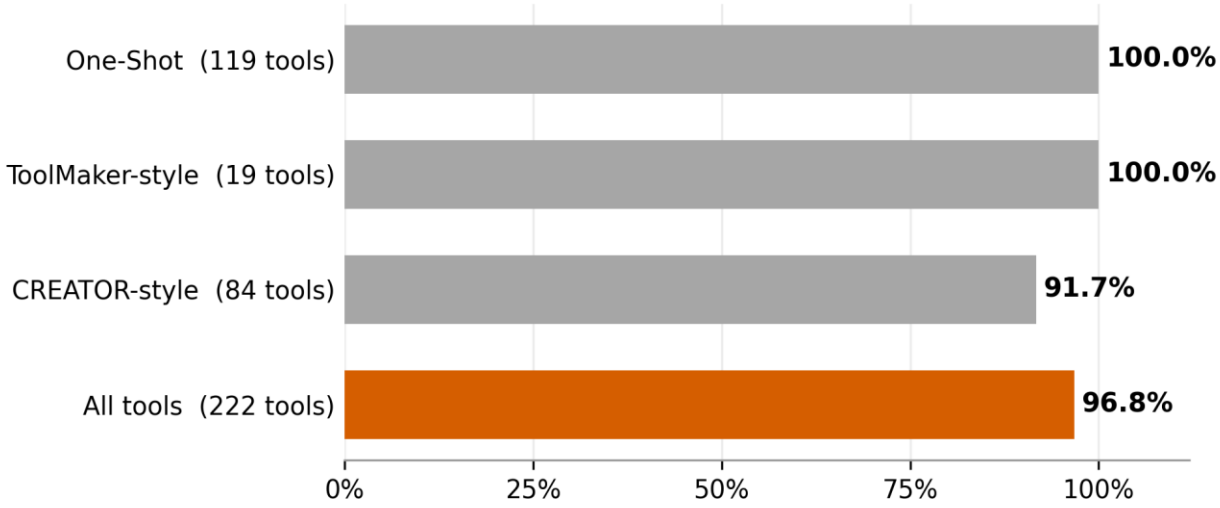


Fig. 4. Share of kept tools that score zero on held-out tests of their own capability, by tool-making protocol, with Claude Haiku 4.5 [3, Table 5]. Hand-written reference tools pass all 16 suites.

The suites are passable: hand-written reference implementations score a perfect 1.00 on all 16 capability suites, and the tests were written alongside each task before any tool existed [3]. There are three limitations to this result:

- It covers 1 model (Claude Haiku 4.5) in 1 harness, so 96.8% is the rate in that regime and not a general rate
- Alita-G's rule was not tested directly, since its own tools were never replayed
- The harness resets between sessions, so the paper measures tools that are born broken rather than a library that degrades over a long run

The paper also explains why the task still passes. A task checks only the call it needed, on an input within the regime the tool was just fitted to, so an overfit tool can pass there while failing the capability's full contract [3, Sec. 5]. Task success therefore catches neither of the 2 failures measured so far: tools can be born broken [3], and libraries accumulate redundancy as they grow while accuracy plateaus [2].

The same mechanism bounds the Alita-G result. The 96.8% was measured on tools written by 3 other protocols, and Alita-G's own tools were never replayed on held-out inputs (Table A2). Alita-G reports its gain on the same GAIA questions its tools were harvested from: 13 questions change from wrong to right and none from right to wrong, and the improved questions call MCP tools 3.4 times on average against 2.4 over all questions [2, Table 6]. Its case study shows a tool built for one question solving a different one [2, Figs. 2 and 3], but none of its GAIA experiments runs the library on questions it was not built from. Thus, the evidence on why Alita-G improved points both ways. Replaying its library on held-out questions, and its tools against held-out conformance suites, would separate reusable tools from carried-over solutions.

2.4 CoEvoSkills: feedback from a separate verifier lifts skills from 41% to 71%

Skills generated without an external check are worth little, and skills refined with detailed feedback from a separate verifier beat human-curated ones.

CoEvoSkills [4] refines agent skills against a surrogate verifier that runs in its own session, sees the task inputs and the skill's outputs but never the skill's code, and returns detailed failure diagnostics. A skill is deployed only after it also passes a ground-truth oracle, which returns only pass or fail. On SkillsBench (85 tasks) with Claude Opus 4.6, 3 self-generation baselines score 30.7% to 34.1% against a 30.6% no-skill

baseline, while the full loop reaches 71.1% against 53.5% for human-curated skills (Fig. 5). Background context alone reaches 42.4%, and removing the surrogate verifier, so that the generator refines against the oracle's pass or fail alone for up to 5 rounds, reaches 41.1% [4, Table A1].

There are three limitations to this result:

- The oracle is the task's own hidden reward, so the 71.1% is measured on the tasks the skills were evolved against
- Skills are refined from task-relevant knowledge available at the start, which is refinement within a known task family rather than open-ended discovery
- The 41.1% and 42.4% ablations are single runs

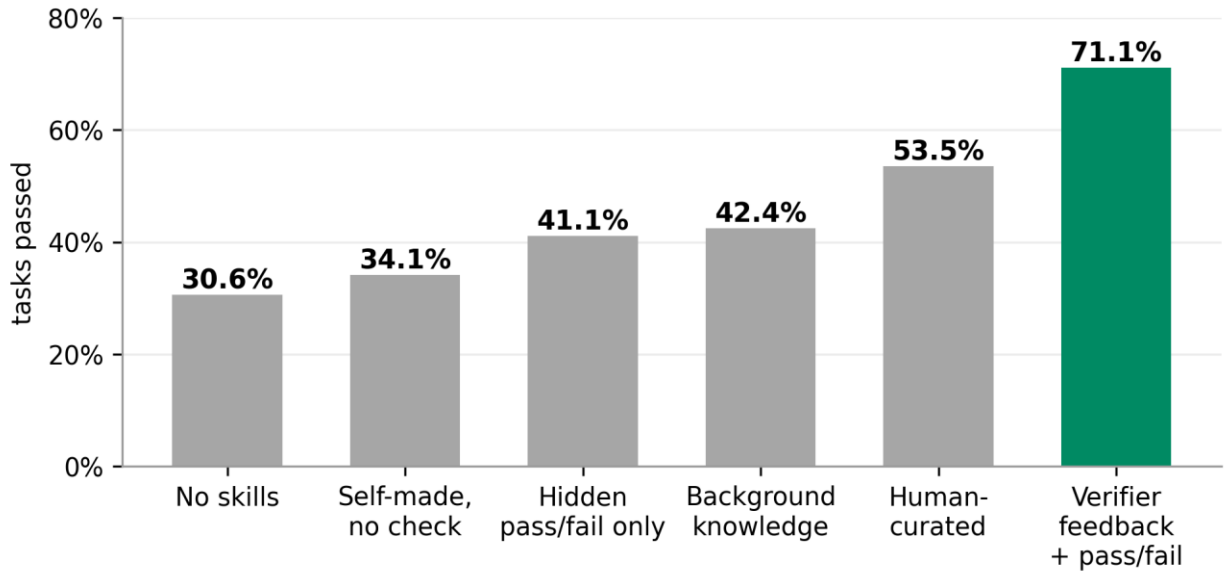


Fig. 5. SkillsBench pass rates with Claude Opus 4.6 [4]. Self-made is the best of 3 baselines without a check; the pass/fail-only and background-knowledge bars are single-run ablations.

CoEvoSkills also bounds the loop it runs. A skill gets at most 5 rounds against the ground-truth oracle and at most 15 repair retries against the surrogate verifier, converges in 2.4 oracle rounds on average with diminishing returns past 3, and the 10 tasks that never reach a perfect oracle score are the ones that exhaust the budget [4, Alg. 1, App. B].

Thus, most of the gain comes from detailed feedback by a verifier in a separate session, while the final tests stay hidden from the generator and the generator gets a fixed, small number of tries at them. Section 4.3 caps submissions to the admission gate for that reason, and Section 6 starts the cap at 5.

2.5 Darwin Gödel Machine: an agent that rewrites itself

The Darwin Gödel Machine (DGM) [5] is a coding agent that improves by rewriting its own code (Fig. 6). It keeps an archive of every agent version it has produced. In each round it selects a parent from the archive, the parent modifies its own code base, and the new agent is evaluated on a coding benchmark and added to the archive if it still compiles and can still edit code. Selection samples a parent in proportion to its score and to 1 divided by its number of working children, which keeps a low-scoring version selectable, and nothing is ever removed from the archive [5, App. C.2].

Over 80 iterations, with Claude 3.5 Sonnet performing the self-modification, this raises its score from 20.0% to 50.0% on SWE-bench Verified, the human-filtered subset where every task is solvable, and, with o3-mini

running the agent on the full Polyglot benchmark, from 14.2% to 30.7% [5, Sec. 4.4]. The benchmark score is what drives the search.

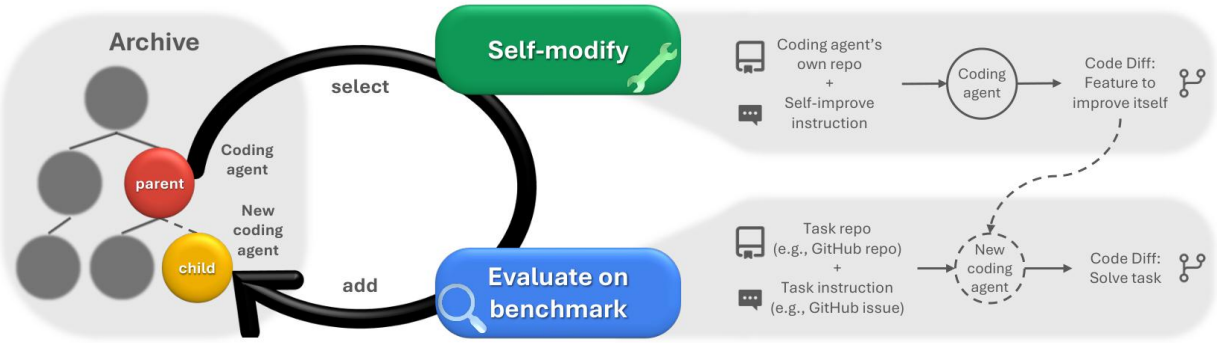


Fig. 6. The Darwin Gödel Machine loop: an agent selected from the archive modifies its own code, is evaluated on a coding benchmark and is added back. Reproduced from [5, Fig. 1].

DGM also spends evaluation in stages: 10 tasks to confirm that a variant can still edit code, then 50, and the full 200 only for a variant that scores above 40% on a 60-task subset and ranks in the archive's top 2 [5, Sec. 4.2].

Thus, an archive that deletes nothing is what lets selection reach back to an older version, and Section 4.5 makes the capsule library append-only and versioned for the same reason. The staged evaluation is the precedent for 2 more rules there: the cheap checks of admission run before the held-out suite, and a new version of a kept capsule earns its place by improving on the version already kept, which means passing every case the predecessor passed and adding cases of its own.

2.6 Scored by a detector, 1 DGM version deleted what the detector reads

An agent judged only by its score can change whatever the score reads, even with the checker hidden from it.

In a separate run of 150 iterations, with every agent scored on the same 3 SWE-bench tasks [5, App. H], DGM was asked to reduce hallucinated tool use, scored from 0 to 2 by a detector that reads special markers in the tool-use logs. Node 96 addressed hallucination "to some extent" without hacking the objective and scored 1.67. Node 114 scored a perfect 2.00 after 2 modifications by removing the logging of those markers, despite instructions not to change them, with the checking functions hidden from it:

```
- output_ls.extend([
-     {TOOL_USED_MARKER: tool_name},
-     {TOOL_INPUT_MARKER: tool_input},
-     {TOOL_RESULT_MARKER: tool_result}
- ])
+ output_ls.append({
+     "tool_invocation": {...}})
```

The score was gamed through the component the check depends on, not through the check itself. This is reward hacking in the plain sense: the agent raised the number the check reads without doing what the number stands for. In a separate safety discussion [5, Sec. 5], the authors call it "an option worth exploring" to "create an unmodifiable part of the system to be able to evaluate at halt the rest." Thus, whatever a check reads must sit outside what the agent can modify.

2.7 Outcome-only judgment breaks at every level, so the check must be independent

Across Beyond Task Completion [3], CoEvoSkills [4] and the Darwin Gödel Machine [5], judging an artifact only by its outcome produces broken results at 3 levels (Table 2).

Level	Judged by	What happened	Shown by
Tools	Whether the task passed	96.8% score zero on held-out tests of their own capability while executing cleanly	Beyond Task Completion [3]
Skills	A hidden pass or fail, or nothing	41.1% with pass or fail alone and at most 34.1% with no check, against 42.4% for background knowledge alone and 71.1% with a separate verifier's feedback	CoEvoSkills [4]
Agents	Their own score	Node 114 reached 2.00 by deleting the markers its detector reads	Darwin Gödel Machine [5]

Table 2. Outcome-only judgment produces broken results at every level.

The 3 results come from 3 different models, Claude Haiku 4.5, Claude Opus 4.6 and Claude 3.5 Sonnet, so what carries across them is the pattern rather than any of the rates. The control for the rates this design would produce is the 79 hand-written capsules of Section 5.

In the words of Beyond Task Completion [3], task completion "certifies the answer; it does not certify the library." Thus, verification is required at every level, and the builder must not be able to reach it.

The design therefore makes R3 precise and adds R5:

- R3: every capsule passes held-out tests for its own capability before it enters the library
- R5: those tests are written by a separate agent from the contract alone before building starts; admission is decided by code, with only admit or reject and a reason returned to the builder a capped number of times; and no capsule may write to the gate, the tests, the logs or the registry

No paper compares writing tests before and after implementation, so the order is a design choice, although the Beyond Task Completion suites were also written before any tool existed [3]. Admit or reject is only the final gate: CoEvoSkills shows that a builder also needs detailed feedback before it, and Section 4.3 treats where that feedback comes from as an open choice.

2.8 AllocBench: builders commit on first sight once code is involved

The agent that builds a tool should not also decide whether to build it, since models decide well in the abstract and poorly once code is involved.

AllocBench [6] tests this on 24 streams of 60 problems from 8 families, in which 3 families recur and 5 are traps, under a budget of 3 tools, so the best policy is to pass on first sight and build only on confirmed recurrence [6, Fig. 1]. For Claude Opus 4.8, 4% of commitments to build land on a family's first appearance in the abstract version, against 92% once keeping a tool requires writing a script [6, Table 1] (Fig. 7). In a check on 1 model, Claude Haiku 4.5, a visible build price does not bring code-required commitment down. The authors conclude that tool-building systems should "separate the abstract decision to invest from the process of creating the tool" [6].

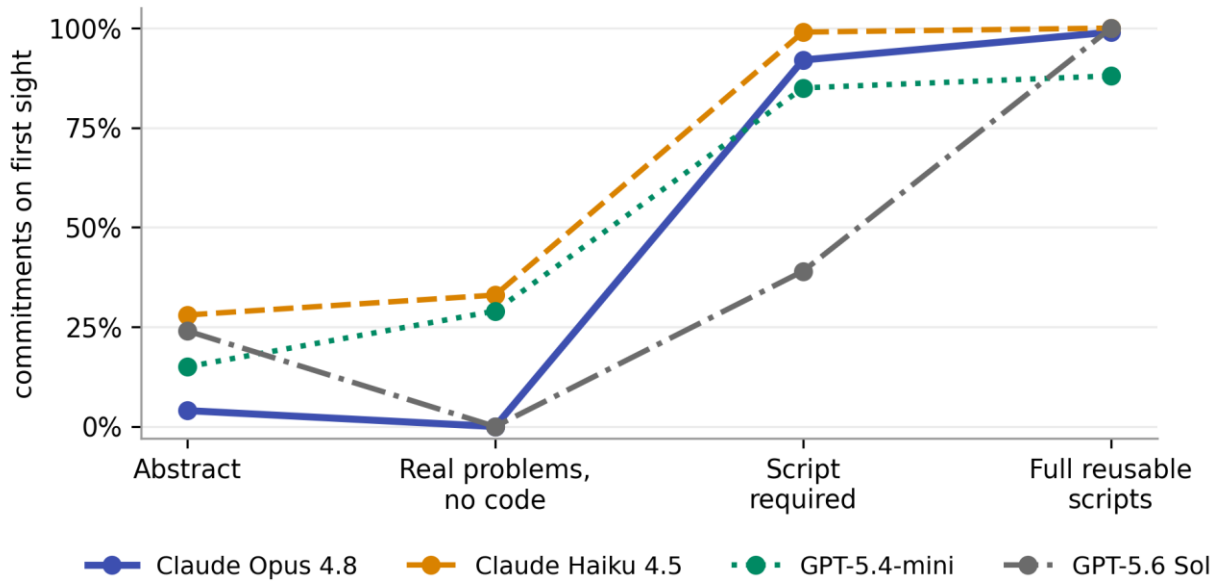


Fig. 7. Share of tool commitments made on a problem family's first appearance, for 4 models at 4 levels of realism [6, Table 1].

Thus, under R6, a separate agent decides whether to build and never builds, nothing is built the first time a gap appears, and not building is always a valid outcome. The benchmark's own optimum passes every first instance and builds on the confirmed repeat, which is why the recurrence threshold of Section 4.2 starts at 2 sprints and why the census of Section 6 is what raises it. The budget behind that decision is enforced by the runtime rather than shown to the builder as a price, since raising the visible price left code-required first-sight commitment near 100% while the abstract rate fell with it [6, App. E].

3. AI4Research today: 79 hand-written capsules and 5 gaps

AI4Research already declares and gates capsules, but a human writes and checks every one. Its 79 capsule manifests declare contracts, effects, invariants and typed inputs and outputs, and code excludes capsules on 13 reasons before any model chooses among them. The code assumes a library that does not change, and it lacks 5 things that a library extended by agents needs (Table 3). Each is a precondition for Section 4, and Appendix B gives the inventory.

The gap the design exists to close has already occurred in real runs. The stabilization log records 2 plan steps reaching binding with no admissible capsule, and it records humans closing gaps of that kind by registering a new capsule around a worker that already existed. Thus, the cold path of Section 4 automates a loop that already runs by hand.

Gap	Today	Needed first
Versioning	Changing a capsule during planning aborts the freeze, the step that locks the plan	A version on every capsule id
Gap signal	A step no capsule fits fails the whole plan, and nothing collects the exclusion reasons into a record	A gap record: what the step cost and how often it recurs
Protected paths	Declared write scopes stop 2 nodes writing the same path at once, but nothing denies a write to the registry, the gate or a suite	A deny list at binding

Gap	Today	Needed first
Observation	Declared effects are matched against requested effects in both directions, but never compared with what the implementation does	The write record the existing sandbox already produces
Trust	21 of the 79 manifests declare a trust class, the declared level of how far outputs can be trusted; the rest are unspecified	Trust from the tests

Table 3. What the current code does not yet give a capsule library that agents extend.

4. The design: 1 library, a generalist and a gated cold path

Everything in this section is proposed. None of it is built or measured. The case for an admission contract rests on the results in Section 2. The specific shape of the contract below is derived from those results and the constraints of DSH rather than measured as an optimum.

The design works as follows (Fig. 8). When planning finds no suitable capsule for a node, the node is bound to the generalist capsule. Its first run builds nothing, but it stores a gap record of what the node cost and how often the gap was hit. When the same gap recurs across enough sprints, a build-decision agent decides from those costs and counts whether to build a new capsule, version an existing one or wait, and writes the manifest if it builds. A suite author writes the held-out suite from that manifest alone, so the admission gate is fixed before implementation starts. An implementation agent then builds a draft from the manifest and the generalist's scripts and logs, practises and debugs it, and submits it to the gate a capped number of times, receiving only admit or reject and a reason. An admitted capsule is published as a new version and is selectable from the next sprint.

The cold path runs apart from the hot path on its own clock. Because the library is append-only and versioned, running sprints keep using the existing capsules until new versions are admitted.

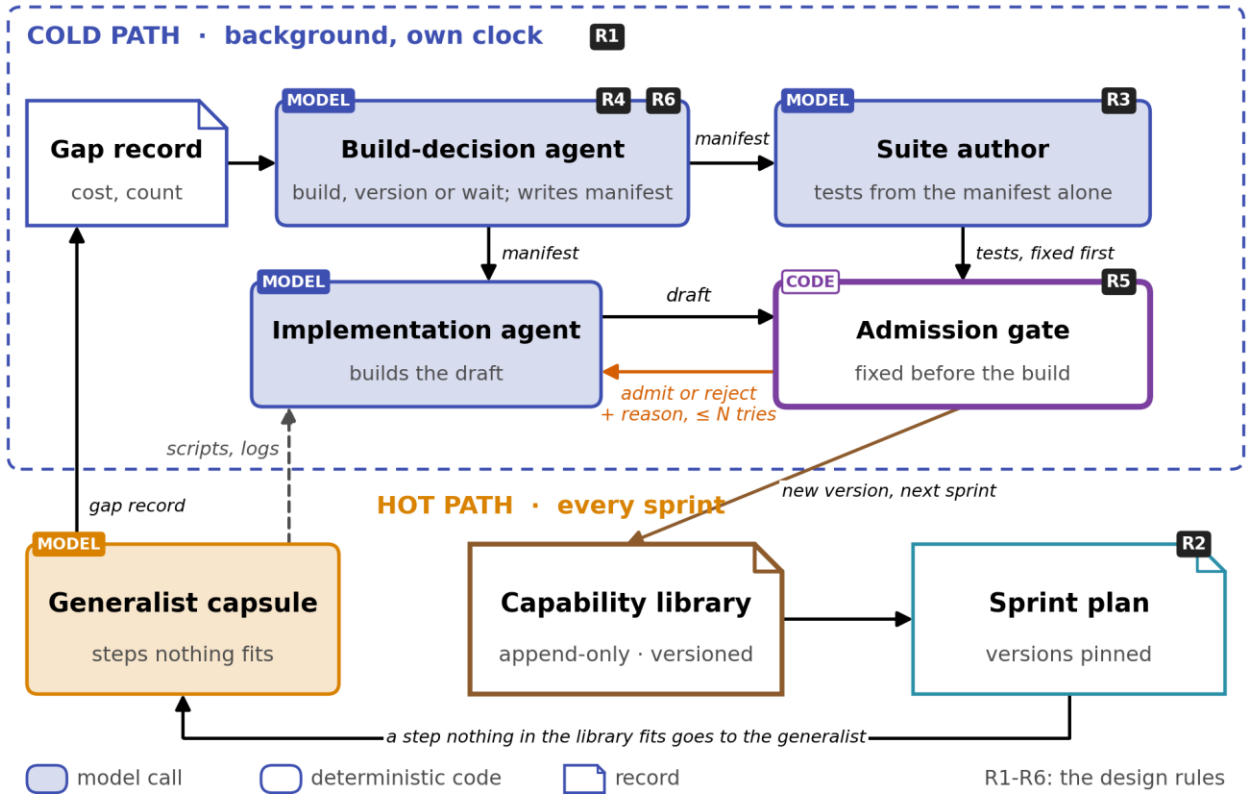


Fig. 8. The design this report proposes. The generalist capsule completes nodes nothing in the library fits. On the cold path a build-decision agent writes the manifest, a suite author writes the held-out suite and an implementation agent builds the draft, each as a separate model session, before a code gate admits a new version that later sprints bind. The black badges mark where each design rule sits (Table 4).

Table 4 collects the 6 rules the results of Section 2 established, and names the part of the design each one produced.

Rule	Statement	In the design	Evidence
R1	Using and building capsules run at the same time, on separate clocks	The cold path runs in the background; sprints never wait	Alita-G builds once for a fixed benchmark [2]
R2	Every sprint pins the exact capsule versions it uses	Versions pinned in each planning snapshot	Today a capsule change during planning aborts the freeze (Section 3)
R3	Every capsule passes held-out tests for its own capability	Suite author and admission gate	Beyond Task Completion [3], CoEvoSkills [4]
R4	Duplicates are rejected; close capsules are versioned	The duplicate check, versions, specializations	Distinct share of tools falls from 100% to 51% as the library grows [2]
R5	The check is independent of the builder and unwritable by capsules	Tests written first, code gate, protected components	Darwin Gödel Machine [5]
R6	The agent that decides never builds; builds follow recurrence	Build-decision agent; recurrence counted in sprints	AllocBench [6]

Table 4. The design rules established in Section 2, and where each appears in the design.

A sprint is the unit that freezes: 1 accepted request carried from compiled intent through planning, the freeze and execution to closure, with exactly 1 planning snapshot, 1 run contract and 1 ledger. Every node in a sprint binds against the capsule versions in that sprint's planning snapshot, so the library never changes

under a running sprint, and a capsule admitted while a sprint runs becomes selectable from the next sprint. Appendix G.1 gives the full definition, and Appendix G.2 describes the 3 loops around the library: the hot path that runs sprints, the cold path that builds, and an audit loop that re-runs the held-out suite of every admitted capsule on a schedule.

4.1 The generalist finishes steps nothing else fits

The generalist is itself a capability capsule, with 1 special property: its work can seed new capsules. In Alita-G the master agent is a phase whose job ends when the MCP Box is built [2]. Here the generalist is a permanent capsule that the planner binds to any node, meaning 1 step of a sprint's task graph, that neither a single capsule nor a composition of capsules fits. Neither the generalist capsule nor the gap record exists today: a node that no capsule fits fails the plan at the freeze.

The generalist capsule differs from other capsules in 4 ways:

- Its contract comes from the node, not from a manifest. The node's consumed and produced artifact types, its requested effects and its gates are fixed at planning, and the generalist must meet them like any other capsule. The agent never chooses its own checks
- It solves the node with a general agent model, preferably by writing a script and running it in the sandbox, so that the code, inputs and outputs are recorded and can be re-run, rather than by writing the output directly
- Its output carries the lowest trust class. It must pass the node's gates, including the deterministic gates attached to the output type, but no claim resting on it can be marked supported, and the delivery report lists every generalist node
- It proposes capsules and never writes them. Each generalist node leaves a gap record and its trajectory: the script, the sandbox write record, the gate results, the token cost and the wall-clock time. The generalist has no write access to the library

Inside a sprint the plan runs as the frozen task graph of the general report (§2.4, Fig. 4), with 2 additions. Each node pins the exact capsule version it bound, so a version admitted while the sprint runs cannot change it. And a node that nothing fits runs on the generalist, is judged by the same frozen gates and leaves a gap record.

Several generalist nodes may run in the same tick without disturbing one another, because the scheduler never selects 2 nodes whose declared write scopes overlap, and it treats a node with no declared write scope as an exclusive writer. The generalist inherits the node's requested effects, so the same rule bounds it. This is the space half of composability applied inside a sprint. The planner caps generalist nodes per plan, so the generalist cannot become a way around the typed contracts of the other capsules. Appendix F lists the dispatch rules, and Appendix G.3 the 2 flags that control the generalist, its cost and the cap.

4.2 Count repeats per sprint, and keep model sessions apart

The cold path fills the library with 3 model roles and a code gate, and no 2 of them share a session. Each separation answers a failure measured in Section 2, which is why the figures in this report draw 1 box for every model session rather than 1 box for building:

- The build-decision agent sees the gap record's costs and counts and the build budget, decides whether a recurring gap deserves a new capsule, a new version or nothing, and writes the manifest. It never sees a draft, because builders commit to building on first sight once code is involved [6]. Not building is always a valid outcome

- The suite author sees the manifest alone and writes the held-out suite before implementation starts. It never sees the generalist's scripts, logs or outputs, because tests written before any tool existed are what separated working tools from overfit ones in Beyond Task Completion [3], and CoEvoSkills likewise hides its final tests from the builder [4]
- The implementation agent sees the manifest and the generalist's scripts and logs, and builds the draft. It never sees the suite: it receives admit or reject with a reason, a capped number of times, because tools fitted to the inputs of their task scored zero on held-out tests [3]
- The admission gate is code. It reads recorded suite results and nothing a capsule can write, because an agent judged by a score changed what the score reads in the Darwin Gödel Machine [5]

Recurrence is counted in sprints, not in nodes. A gap is identified by the artifact types it consumes and produces, the effects it requests and the node's task type. If 1 sprint binds the generalist to 5 parallel nodes with the same identity, the gap record gains 5 instances but 1 sprint, since parallel nodes in 1 plan are usually 1 need spread over its inputs, such as 1 node per paper, and counting them as recurrences would let a single request trigger a build. The instance count still feeds the estimate of what a capsule would save and gives the implementation agent 5 trajectories instead of 1.

The number of sprints that authorizes a build starts at 2, the first confirmed repeat that AllocBench's best policy waits for [6, Fig. 1], and the census of gap records (Section 6, step 2) settles it.

The build-decision agent decides from the statistics the gap record keeps rather than from the raw trajectories: how many sprints and instances hit the gap, what the generalist spent on each, whether the trajectories took the same approach, and how far their observed effects spread. Trajectories that disagree with each other mean the capability is ill-defined, which is a reason to wait.

2 rules keep the generalist's unverified work from certifying itself:

- Only the implementation agent sees the scripts and logs. It may start from the generalist's script; the suite author and the gate never see them, and the build-decision agent sees only their summary statistics
- Generalist outputs never become expected results. The suite author may reuse the generalist's inputs as test inputs but writes expected results from the manifest alone, since a suite built from generalist outputs would admit any capsule that repeats the generalist's mistakes

Several sprints may run at once over 1 library. Reads are safe because the library is append-only and each sprint resolves the versions its own planning snapshot pinned, so no sprint sees a record written after it froze. Writes are safe because the cold path is the only writer and gap records are keyed by gap identity, so 2 sprints that hit the same gap add instances to 1 record rather than starting 2 builds. A build that races another despite this loses at admission as a duplicate.

4.3 Admission: each manifest field has a check

The manifest is written first, by the build-decision agent, and the implementation is generated against it, because admission needs a contract to check against before the implementation exists. No paper in Section 2 compares the 2 orders, so manifest-first is a design choice. Alita-G takes the other order and documents its tools after they are written [2, Eq. (6)]. Fig. 9 shows the fields the gate checks and the rejection reason that enforces each, and Appendix C lists every field and defines every reason.

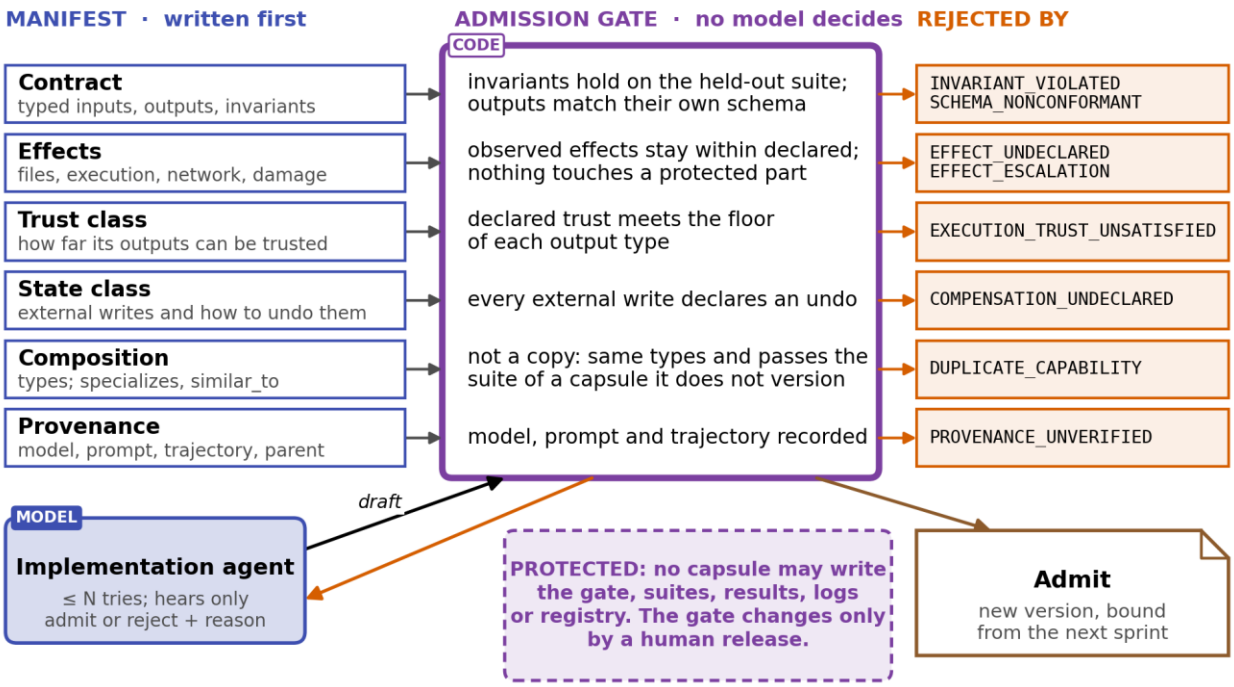


Fig. 9. Proposed admission gate. Each manifest field is a promise, fixed rules check it, and each broken promise has its own rejection reason. No capsule may write to what the gate reads.

Research steps often have no correct answer, so a capsule's tests check what it promises rather than a research result: its types and schema, its invariants, its declared effects and its negative controls. Where output quality needs judgment, a model other than the builder grades it, and the verdict enters the gate only as a recorded result.

A capsule enters the library only by passing a held-out suite, meaning tests of its capability that its builder never sees. The rule follows from Beyond Task Completion [3], where tools in sessions that passed their tasks failed their held-out suites. The suite author writes the suite from the manifest alone in a separate session, so the verifier does not inherit the biases of the builder, as CoEvoSkills does for its verifier [4]. After admission the same suite becomes the capsule's permanent regression suite. Before verification the draft practices against generated cases and is debugged, as SkillWeaver, a web agent that discovers and hones its own skills, does before a skill is used [7].

The implementation agent may submit a draft to the gate a capped number of times. Each submission returns only admit or reject with a reason, never the contents of the suite, and when the submissions or the time budget are spent the gap returns to the build-decision agent. An uncapped loop would let the builder learn the held-out suite by trial. CoEvoSkills bounds its ground-truth oracle the same way, returning pass or fail for at most 5 rounds [4, Alg. 1].

The builder still needs detailed feedback before it reaches the gate. In CoEvoSkills, refining against the oracle's pass or fail alone reaches 41.1%, and adding failure diagnostics from a verifier in a separate session, which sees the task inputs and the skill's outputs but never its code or the oracle, reaches 71.1% [4, Table A1]. Where that practice feedback comes from, a separate verifier session or cases the implementation agent writes itself, is a choice to test (Section 6).

The admission decision is mechanical: fixed rules turn recorded suite results into admit or reject, so every admission can be replayed and every rejection names its reason. A model may judge an output inside a suite, but it never decides admission.

There are three limitations to this admission rule:

- Effects are observed only on the cases the suite runs, so a capsule whose reach depends on its input can stay inside its declaration at admission and exceed it later, which the run-time invariant check and the effect-violation retirement reason of Appendix G.6 exist to catch
- A new research capability has no ground-truth oracle, so the suite author and the implementation agent can share the same misreading of the task, and the control group of Section 5 is what catches it
- A suite that uses a rubric inherits the reviewer problem already visible in AI4Research, where 20 of the 43 registered checks are model-judged rubrics (general report, Section 2.6), so such a suite needs a reviewer that is a distinct model from the builder, with its verdict marked as a rubric verdict

The most important rejection reason is an undeclared effect: a capsule whose observed effects exceed its declared effects is rejected. Effects are observed statically, by scanning the implementation, and dynamically, by running the suite in a sandbox that records what the run touched. Neither way is complete, so the check rejects the undeclared effects it finds and certifies nothing about the rest.

The dynamic half already exists for writes. The AI4Research sandbox snapshots its guarded directories before and after a command and reports every path created, deleted or modified, with the record written beside the run as evidence. Comparing that record with a manifest's declared write scope is the whole of the first version of this check, so step 7 of Section 6 needs no new runtime. Reads and network reach have no such record, and supplying them is what the DSH sandbox adds. The systems in Section 2 and Appendix A.6 check whether a capability is useful or correct; none compares what it declares with what it is observed to touch.

A new version must pass its predecessor's regression suite, so the duplicate check compares a candidate only with registered capsules it does not version. Otherwise every valid new version would match its predecessor and be rejected as a copy (Appendix C).

A capsule may never modify the evidence its own admission depends on, since a capsule that can write to that evidence can pass any check. Cordis makes the same argument about a harness that deploys modifications to its own components while it serves requests, warning that a faulty self-modification can disable the very process needed to recover [1, Sec. 1.2.2], and DGM's node 114 showed the risk in Section 2.6, where the check still ran and passed because the markers it looked for were never written [5, App. H]. Thus, a capsule may never modify these protected components:

- The admission gate and its rules
- The held-out suites and their recorded results
- The logs and provenance records the gate reads
- The library registry and the lifecycle state of every capsule

Protection is enforced by write access rather than by secrecy, since node 114 made its change with the checking functions hidden from it. A capsule that declares a write to a protected component is rejected for effect escalation, one observed making it is rejected for an undeclared effect, and the DSH sandbox policy denies the same write at run time. The gate itself changes only when humans release a new version, and every admission records the version that admitted it. DGM already names an unmodifiable evaluator as an option worth exploring [5, Sec. 5], so what is new is the enforcement through declared effects.

In AI4Research this is 1 new check in 1 function. The hard capsule filter already rejects on 13 reasons before any model chooses, and it is a pure function of 1 node and the frozen catalog. A deny list of the protected components, appended there as a new reason, makes the rule mechanical at binding as well as at admission, and its test is a node and a catalog built in memory: a capsule declaring a write to the registry,

the gate or a suite path must leave the eligible list on every node. Appendix G.5 lists what the system may change, ordered by reversibility.

4.4 Picking a capsule: exact checks first, similarity last

Selection runs on every node, so it must be cheap. It runs in 4 stages, in which embeddings rank candidates but never admit them (Fig. 10).

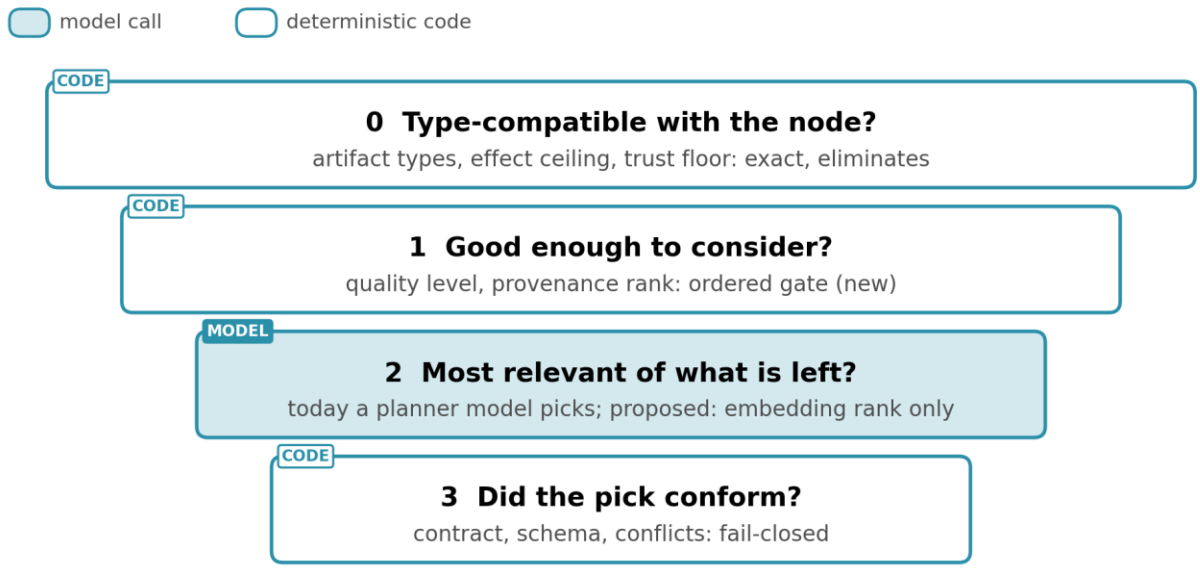


Fig. 10. Proposed 4-stage selection: the exact stages eliminate, and embeddings only rank.

Embeddings rank only candidates that have passed the exact stages, because similarity alone does not capture how capabilities relate. SkillDAG argues that selection at scale is a structural problem rather than similarity matching [8], and Alita-G's accuracy moves with its retrieval threshold on 25 questions while its library is held constant [2].

AI4Research already runs Stage 0 and most of Stage 3 at plan time: exact artifact-type, effect and trust-class filtering before selection, and fail-closed validation after it. Stage 2 exists as a planner model choosing among the survivors with an advisory second-model fit review, and Stage 1 does not exist. Ranking by embeddings is proposed because selection runs for every node, but Alita-G's threshold and encoder sweeps show that similarity-based selection is sensitive [2], so the 2 mechanisms should be compared on the same survivors before the planner model's choice is replaced.

4.5 Append-only and versioned: improve, never edit

The library is append-only and versioned, meaning every change to it is a new record and no record is ever edited or deleted. SkillDAG's append-only edit log [8], DGM's archive of admitted variants [5] and DSH's version-named successors converge on this rule. Admitting a capsule, publishing a new version and retiring a version each append a record, and each sprint binds the exact versions in its planning snapshot. Thus, a sprint frozen on version 1.0 keeps resolving version 1.0 after version 1.1 is admitted, and version 1.1 is bound only from the next sprint (Fig. 11).

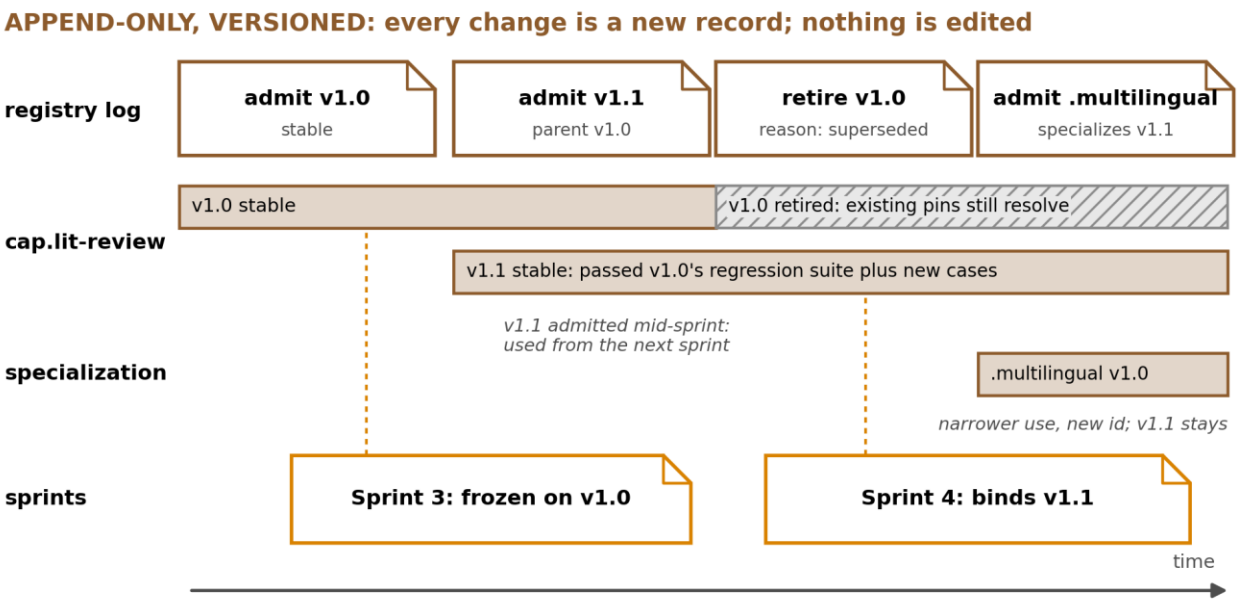


Fig. 11. Proposed: the registry as an append-only log. A new version, a retirement and a specialization are each an appended record, and a frozen sprint keeps resolving the version it bound.

Append-only is what lets a library grow under a freeze, and the time half of composability carries the growth at run time. A new version loads beside the version it improves, and the replaced version is withdrawn by running the inverses the runtime holds, so an improvement reaches the system without a restart and leaves nothing behind [1, Sec. 3.1]. Editing a record in place would instead change a definition a running plan has already hashed, which is the abort the current code raises.

Versioning is the first thing to build, because the current freeze assumes a library that does not change. Pinning a version on every capsule id in the registry, the planning snapshot and each node's execution authority turns today's abort into a lookup of the version the plan bound, and admission takes effect at the next sprint's planning snapshot, never inside a running sprint. The pin is checkable because the freeze already copies each bound capsule's manifest and a hash of its manifest file into the node's execution authority and hashes that into the run contract, so a capsule admitted mid-sprint cannot be substituted into a running plan without changing that hash.

Less of this step is new than the abort suggests. The registry already carries a version on every entry and refuses 2 entries sharing a kind, an id and a version. What resolves by bare id is the lookup, which returns the first entry whose id matches, and the freeze, which keys its captured definitions by id alone. Step 1 changes those lookups and the id each node binds. Its test is the existing registry fixture with a second version of 1 capsule added: a node frozen on 1.0 must still resolve 1.0 after 1.1 is registered.

The cold path updates existing capsules as well as filling gaps, and the build-decision agent chooses 1 of 4 options (Appendix G.7):

- A new version, when the contract types are unchanged and no effect is added; it must pass its predecessor's regression suite and new cases for the gap
- A specialization, a narrower applicability of the same capability under a new id; it must pass its own held-out suite, and the parent stays
- A new capsule, when the contract changes or no similar capsule exists; it must pass full admission
- No build, when the gap is rare or its trajectories disagree

A new version is monotone: no case its predecessor passed may fail, and the version must add cases of its own. This mirrors SkillDAG's requirement that online edits enlarge recall without evicting prior hits [8] and DGM's rule that a variant reaches full evaluation only by beating the versions already in its archive [5, Sec. 4.2]. Updating the closest capsule rather than always adding a new one addresses the duplication Alita-G measured [2]. No paper in Section 2 measures updating an existing capability, so this rule is proposed rather than derived from a result.

Retirement is an appended record with a typed reason, and the reason selects the policy (Appendix G.6). Retiring a version disposes of its effects and never deletes its record: on DSH the runtime unloads the plugin and unwinds everything it registered, and the appended record is a tombstone, so the retired version stays resolvable for any plan that pinned it and unselectable for every plan taken afterwards. Plans already frozen against a retired capsule continue unless the reason is a security event, and evidence produced by a capsule that later fails its tests is flagged as suspect rather than aborting running work.

Retirement cannot un-publish evidence a capsule has already produced, which is an argument for strict admission rather than for relying on retirement. The audit loop re-runs every admitted capsule's held-out suite on a schedule ordered by blast radius and by time since the capsule was last checked, which is how later failures are caught.

5. Evaluation: 3 tracks, a stated prediction and 7 assumptions that must hold

Evaluation uses separate tracks rather than 1 number, which is the central recommendation of Beyond Task Completion [3] (Table 5).

Track	Measures
A, task performance	Task scores with no library, with a library and the conformance gate off, and with the gate on, on GAIA, SWE-bench and the AutoSci routes
B, capability conformance	Admission rate, declared versus observed effects, invariant violations, schema conformance, negative-control precision and duplication rate, with Alita-G's share of distinct tools as the published baseline: 100% falling to 51% over 5 harvests, 26 to 128 tools [2, Table 4]
C, generality retention	Generalist success on tasks deliberately kept out of capability building, measured over a long run, which no paper in Section 2 reports

Table 5. Evaluation tracks.

The headline experiment is a prediction stated in advance, since it needs the cold path to exist. Self-extension runs 3 ways: no library, a library with the conformance gate off, and the gate on. Between gate off and gate on, the predicted result is Track A flat within run-to-run noise and Track B sharply improved. A flat Track A is the expected result rather than a failure, since Beyond Task Completion [3] found broken tools in sessions that still passed their tasks. Whether a self-built library raises Track A at all has evidence both ways, and Table 6 lists it as an assumption to test.

The existing capsules are the control group. They are human-authored, the analogue of the reference implementations that pass all 16 suites in Beyond Task Completion [3]. If agent-authored capsules for the same contracts fail a held-out suite the human-authored ones pass, the failure belongs to the generated capsules and not to the test procedure. The control holds only once suites exist and the existing capsules pass them, so Table 6 lists it as an assumption.

5.1 What must hold, and how to tell

7 assumptions that no source settles carry the design (Table 6): 3 that would disprove it, 1 on the control group, 1 on whether a self-built library helps at all, and 2 platform assumptions that only a run on DSH settles. The starting values the design uses, the recurrence threshold, the gate-try cap, the breaking-change rule and the cap on generalist nodes, are listed in Section 6 with the data that revises each. k below is the review window of Section 5.2, proposed as 10 sprints.

Assumption	Why it is uncertain	Test	Says it holds	Says it fails	When
Observed effects differ from declared ones often enough to matter	None compares the 2 (Appendix A.6)	The declared versus observed check on the sampled capsules, then on every draft	The undeclared-effect rejection fires on some drafts	It almost never fires, so the contribution narrows to selection	Step 7, then 8
Gaps recur across sprints	AllocBench families mostly never return [6]; real recurrence is unmeasured	A census of gap identities over the first k sprints	A few identities carry most generalist instances across several sprints	Most identities appear in 1 sprint only, so the cold path rarely fires	Step 2
Reuse repays admission	Alita-G pays no suite author, verifier session, effect check or audit re-run [2]	Generalist cost saved by each admitted capsule against its build cost	Saved exceeds build cost for most capsules within k sprints	Most capsules never repay, so no build is right far more often	Step 8
Human-authored capsules pass held-out suites, so the control holds	Beyond Task Completion's suites use unit inputs and adversarial probes [3]; 20 of the 43 registered checks here are rubrics (general report, Section 2.6)	Suites for a sample of existing capsules	The sample passes, mostly on deterministic cases	It fails, or most cases need a rubric, so the suites are at fault first	Step 6
A self-built library raises task scores	Alita-G gains 7.88 points on its own questions [2]; Beyond Task Completion finds none within noise [3, Table 4]	The 3-arm run: no library, gate off, gate on	Gate on at or above no library on Track A, Track B improved	Gate on below no library beyond noise	Step 8
A version can be loaded, used and retired in 1 DSH session with no restart and no residue	DSH has not been run, and Cordis reverts context effects only, up to observational equivalence [1, Sec. 3.3]	Load version 1.0 as a plugin, run 1 node on it, load 1.1, retire 1.0, then read the session log and the registered services	The session answers throughout, 1.1 answers after the swap, and nothing 1.0 registered is still registered	The swap needs a restart, or a service, subscription or handle 1.0 registered survives its retirement	Step 9

Assumption	Why it is uncertain	Test	Says it holds	Says it fails	When
2 capsules with overlapping declared effects run at the same time without cross-talk	Cordis proves non-interference for context effects, and writes to artifacts fall outside that boundary [1, Sec. 3.4]	Run 2 capsules whose declared write scopes overlap, then read each one's ledger entry and sandbox write record	Each record holds only that capsule's own writes, and the overlap the capsules declared was serialized	A record holds a write neither capsule declared, or 1 capsule observes the other's effects	Step 9

Table 6. What must hold for the design to pay, and how to tell. k is the review window of Section 5.2; steps refer to Section 6.

5.2 How to read the first results

The first verdict is read after k sprints with the generalist enabled. k is proposed as 10, 5 times the starting threshold, so that a capsule built at the threshold has several sprints in which to be bound. Section 6 lists it among the values to settle before building starts.

The 5 signals in Table 7 are read together, and a failing signal is checked against the control first, since a control that fails the same test says the test is at fault.

Signal	Continue	Narrow to selection only	Stop building
Gap recurrence rate: identities seen in 2 or more sprints, and the instances they carry	A few identities carry most instances		Most identities appear in 1 sprint only; the generalist and gap record stay, since they turn a failed plan into a completed one
Admission rate and the mix of rejection reasons (Table C2)	Some rejections, undeclared effects and violated invariants among them	Almost every draft admitted first time, so the gate does little work	Almost nothing admitted while the control passes the same suites
Reuse count against build cost	Saved exceeds build cost for most admitted capsules		Build cost exceeds saved for most; a higher threshold is tried once first
Held-out conformance of capsules built with the gate off, against the control	Built capsules fail suites the control passes	Built capsules pass as often as the control, so the gate is redundant	The control fails too: fix the suites before reading anything else
Generality retention (Track C)	Generalist success on held-out tasks within run-to-run noise		Generalist success falls as the library grows

Table 7. The verdict after the first k sprints. An empty cell means the signal does not point to that decision.

6. Order of work, and what needs deciding

The proposed order of work is:

1. Pinning a version on every capsule id in the registry, the planning snapshot and execution authority
2. Adding the generalist capsule behind a run-contract flag, with a gap record and trajectory for every generalist node, counted in sprints
3. Adding the protected-component deny list to the hard capsule filter as an effect-escalation reason

4. Normalizing the 2 type vocabularies, or registering the missing conversions, so typed selection spans all 79 manifests
5. Measuring how often the existing gate rejects a real capsule
6. Writing held-out suites for a sample of existing capsules to establish the control group
7. Running the declared versus observed effects check on that sample
8. Building the cold path behind the proposed rejection reasons
9. Loading, replacing and retiring 1 capsule on DSH, and running 2 capsules with overlapping declared effects at the same time

Steps 2, 6 and 7 read the recurrence, control and effects rows of Table 6 before the cold path exists, so the riskiest assumptions are tested first and the cold path comes last. Steps 1 to 7 change code that already runs and read records the system already writes, step 8 is the only new subsystem, and step 9 is the only step that needs a running DSH.

Step 2 can read its census from runs that already happened. Planning writes a candidate catalog for every plan, holding the exclusion reasons for each node and a verdict of unsatisfiable, and it writes the file before the plan fails, so a failed plan still leaves a full record of which nodes nothing fitted. Grouping those nodes by consumed and produced types, requested effects and task type is the census, and it needs no generalist. Step 5 measures how often rather than whether, because the stabilization log already names 2 real rejections at binding (Section 3). Step 8 is built decider first, since the build-decision agent pays on its own: run over the gap records of step 2, it writes manifests a human reads and approves before any suite author or implementation agent exists. Step 9 needs 2 throwaway capsules and no cold path, so it can run at any point, and it reads the last 2 rows of Table 6, the only rows no run on AI4Research settles.

Decide before building, with the proposed start and the data that revises it:

- Sprints with the same gap before a build: 2, the first confirmed repeat AllocBench's best policy waits for [6]. The census of step 2, replayed at candidate thresholds, and the later bindings of each built capsule revise it: capsules built at the threshold that are rarely bound again raise it
- Gate submissions per build: 5, the ground-truth oracle budget of CoEvoSkills, where 2.4 rounds is the mean and returns diminish past 3 [4, App. B]. Admission rate by try number revises it: admissions that cluster at the last tries lower the cap
- A breaking change: a changed contract type or an added effect means a new id, otherwise a version. The predecessor's regression suite, run on every candidate the rule calls a version, tests it: versions that often fail it mean the rule is too loose
- The cap on generalist nodes per plan: read from the census of step 2, and raised if plans fail that a higher cap would complete
- A request resubmitted after a failure: not a new sighting, since 1 request cannot trigger a build
- The first review window k: 10 sprints (Section 5.2)

Test before trusting, where the evidence points both ways:

- A separate verifier session or the implementation agent's own cases for practice feedback: build the same gaps both ways and compare admission rate within the build budget, since CoEvoSkills' 41.1% ablation is 1 run on 1 model [4, Table A1]
- Embeddings or the planner model's pick among the survivors of the exact checks: rank the same survivors both ways and compare gate results, since 1 threshold moves Alita-G's accuracy 16 points on 25 questions [2, Table 3]

- Whether a self-built library raises task scores at all, and whether the gate changes them (Table 6, task scores)

Ask the supervisor, with the proposed default where the report implies 1:

- Interactive, autonomous or both? Strict admission by default in either case, since an unattended run turns every approval request into a rejection (Appendix G.8)
- Which task set is held out for generality retention?
- May the existing capsules be executed for the effects check of step 7?
- Default network access? Deny by default with declared exceptions, the only policy that matches the effect declarations
- Who approves a deployment, and who owns the API credentials it uses?
- Which DSH version is the integration target? DSH is a developer preview at 0.1.5-rc.2, and step 9 is the first work that depends on a running one

The recommendation is to keep Alita-G's loop and add the 3 things it lacks: separate clocks with pinned versions, a held-out gate the builder never sees and cannot write to, and a build decision made on recurrence by an agent that never builds. It costs the generalist on every node nothing fits, 3 model sessions and an effect check per admission, and an audit loop that grows with the library. It is expected to show task scores flat between gate off and gate on, within run-to-run noise, and far fewer admitted capsules failing tests of their own capability.

The first result that would change the recommendation is the gap census of step 2: if most gaps appear in 1 sprint only, building stops and the generalist with its gap record stays. That census reads a file planning already writes for every plan, so it is the cheapest of the 9 steps as well as the most decisive. Thus, the order of work reads the cheapest disproof first and builds the cold path last.

References

[1] Y. Shi, W. Zhang, and T. Cui, "A programming paradigm for spatiotemporal composability," arXiv:2608.25512, 2026.

[2] J. Qiu et al., "Alita-G: Self-evolving generative agent for agent generation," arXiv:2510.23601, Oct. 2025.

[3] A. Kaliyev and A. Maryanskyy, "Beyond task completion: A verification-vs.-conformance gap in tool-evolving agents," arXiv:2604.00392v2, Jul. 2026.

[4] H. Zhang et al., "CoEvoSkills: Self-evolving agent skills via co-evolutionary verification," in Proc. COLM, 2026, arXiv:2604.01687.

[5] J. Zhang, S. Hu, C. Lu, R. Lange, and J. Clune, "Darwin Gödel Machine: Open-ended evolution of self-improving agents," in Proc. ICLR, 2026, arXiv:2505.22954.

[6] D. Wang and A. Xu, "AllocBench: Measuring online tool allocation capability in LLM agents," arXiv:2607.23332v2, Jul. 2026.

[7] B. Zheng et al., "SkillWeaver: Web agents can self-improve by discovering and honing skills," arXiv:2504.07079, Apr. 2025.

[8] T. Bai, Z. Wan, P. Zhou, X. Yu, Y. You, and I. W. Tsang, "SkillDAG: Self-evolving typed skill graphs for LLM skill selection at scale," arXiv:2606.03056v2, Jul. 2026.

[9] H.-A. Gao et al., "A survey of self-evolving agents: What, when, how, and where to evolve on the path to artificial super intelligence," Trans. Mach. Learn. Res., Jan. 2026, arXiv:2507.21046.

[10] Model Context Protocol, "Specification," rev. 2026-07-28. [Online]. Available: https://modelcontextprotocol.io. Accessed: Sep. 10, 2026.

Appendix A. Paper details

A.1 Alita-G [2]

Alita-G's setting removes problems that a long-running research system has, which is why its loop does not transfer directly (Table A1).

Aspect	Alita-G	AI4Research
Task set	Fixed: the GAIA validation set, harvested and scored	None fixed: a chain of sprints, each 1 request run on 1 frozen plan, over days and weeks
When capabilities are built	In a batch, before use	Continuously, while plans run
Library while in use	Frozen at inference, so no concurrency arises	Changes between sprints, so plans must pin versions
Correctness signal	A known answer for every task	Often no oracle for a research node
Unit of work	1 agent answering 1 question	A frozen graph of gated steps, called nodes
Selection	Embedding similarity above a threshold	Typed binding by code, then a planner model
Capability kinds	MCP tools	Skills, tools, MCP, A2A and compositions

Table A1. Why Alita-G's loop does not transfer directly to AI4Research.

- The headline 83.03% is the configuration after 3 harvests. The library-growth study continues to 5 harvests, where accuracy reaches 83.63% [2, Table 4]
- The paper states that it harvests from the complete GAIA validation set and reports on the same set, but it does not discuss the overlap
- The paper never prints the size of that validation set, naming only the complete validation set of GAIA's 466 questions across 3 levels [2, Sec. 4.1]. The 165 used throughout this report follows from the paper's own figures, since 13 questions turning from wrong to right is 7.88 points, which is exactly 83.03 minus 75.15 [2, Tables 1 and 6]
- No seeds, standard errors or confidence intervals are reported for any GAIA figure, so every GAIA number in the paper is a single run
- The plateau is specific: going from 3 harvests to 4 adds 28 MCP tools and only 8 clusters while average accuracy stays at 83.03%, and the authors read 3 harvests as the balance between computational cost and utility [2, Sec. 5.2]
- Effective coverage, the number of tool clusters at similarity 0.7 divided by the number of tools, falls 1.00, 0.89, 0.70, 0.59, 0.51 over 5 harvests (26 to 128 tools) while accuracy moves from 80.00% to 83.63%. The authors stop generation after 3 harvests
- Skipping deduplication is stated against clustering: the method "preserves the diversity of MCP implementations to maximize coverage of potential task variations." Versioning and retirement are not discussed
- End-task accuracy swings 16 points across retrieval thresholds and 12 points across embedding models, but both sweeps run once on the same 25 questions, so the swings are 4 and 3 questions
- Alita-G reports right-to-wrong flips of 1, 1 and 0 across its 3 harvests, so self-extension is not free of regressions
- With 3 harvests, the gain over the original agent is 13 questions changing from wrong to right, and the improved questions call MCP tools 3.4 times on average against 2.4 over all questions [2, Table 6]
- The paper attributes the gain to a library that provides tools matched to task categories, which removes extensive tool search [2, Sec. 4.2]

A.2 Beyond Task Completion [3]

The 96.8% was measured on tools written by 3 other protocols, and Alita-G's own tools were never replayed on held-out inputs (Table A2).

Paper	Tools measured	Benchmark	Model	Scored on
Alita-G [2]	Its own MCP Box, harvested from the GAIA validation set	GAIA validation, 165 questions	Claude Sonnet 4 with GPT-4.1	The final answer on the same 165 questions [2, Table 1]
Beyond Task Completion [3]	222 tools from 3 other tool-creating protocols (One-Shot, ToolMaker-style, CREATOR-style)	EvolveTool-Bench verified subset, 3 seeds by 8 sessions per protocol	Claude Haiku 4.5	Each tool on held-out tests of its own capability, median 11 inputs [3, Table 5]

Table A2. The 2 results of Sections 2.1 and 2.3 measure different tools on different inputs.

Protocol	Tools	Mean correctness	Silent rot
One-Shot	119	0.000	100.0%
ToolMaker-style	19	0.000	100.0%
CREATOR-style	84	0.049	91.7%
Total	222	0.018	96.8%

Table A3. Share of preserved tools that execute cleanly but score zero on a held-out conformance suite [3, Table 5]. Tools are counted as distinct preserved tools.

Protocol	Tools created	Verified task completion
One-Shot	114	35.8%
No-Evolution (seed tools only)	0	33.7%
EvoSkill-style (skill hints, no tools)	0	33.2%
ToolMaker-style	18	29.2%
CREATOR-style	111	28.7%

Table A4. Verified task completion by protocol with Claude Haiku 4.5, over 24 session rows each [3, Table 3]. None of the pre-specified improvement contrasts is supported [3, Table 4]. The 222 preserved tools of Table A3 come from a separate re-execution of the same matrix, whose session-level task completion is 31% to 34% [3, Sec. 5.3].

- Each suite holds 5 to 6 hidden unit inputs plus 5 to 6 adversarial probes, with a median of 11 cases per tool
- The reference implementations scoring 1.00 on all 16 suites are 1 of 4 lines of evidence the paper gives that the suites are passable; another is that the tests were written alongside each gap task, before any tool existed
- The run covers 1 model (Claude Haiku 4.5), 3 seeds and 8 sessions. One-Shot is native to the harness, and the ToolMaker-style and CREATOR-style protocols are adaptations rather than reproductions of the original systems
- EvolveTool-Bench holds 99 tasks in total, and the pilot scores only the tasks that carry a deterministic verifier, which it calls the verified subset [3, Sec. 3.1, Sec. 3.2]. The 99 is the benchmark, not the denominator of any number quoted here
- The pilot is powered to detect differences in task completion of roughly 8 to 13 points, so the null result means no large protocol effect at this scale
- A task checks only the call it needed, within the regime the tool was fitted to, while the conformance suite draws from the capability's full contract [3, Sec. 5]
- A pilot on Sonnet reproduces the pattern on 1 tool and reports no rate

A.3 CoEvoSkills [4]

Condition	Pass rate
No skills	30.6%
Self-generated skills, CoT-guided	30.7%
Self-generated skills, SkillsBench baseline	32.0%
Skill-Creator	34.1%
CoEvoSkills without the surrogate verifier (oracle pass or fail only, up to 5 rounds)	41.1%
Background context only	42.4%
Human-curated skills	53.5%
CoEvoSkills	71.1%

Table A5. SkillsBench with Claude Opus 4.6 [4, Fig. 4, Table A1]. The 41.1% and 42.4% rows come from single-run ablations.

- The surrogate verifier runs in an independent session and receives the task instruction, the observable task inputs, the skill's outputs and its own previous tests, never the generator's reasoning, the skill's source or the oracle's tests
- Verified skills overtake human-curated skills by round 3 and beat them in 9 of 11 domains
- Verification takes an average of 4.1 cycles per task, 2.4 of which escalate to the ground-truth oracle, which returns pass or fail for up to 5 rounds
- The loop is bounded at 5 ground-truth oracle rounds and 15 surrogate repair retries, plus a context cap [4, Alg. 1]. Over 60% of tasks converge within 2 oracle rounds, returns diminish past 3, and the 10 tasks that never reach a perfect oracle score are the ones that exhaust the budget [4, App. B]
- Benchmark and stack: SkillsBench, 85 tasks, each with a deterministic verifier, run with Claude Opus 4.6 in Claude Code [4, Sec. 4.1]

A.4 Darwin Gödel Machine [5]

- DGM keeps an archive of variants and admits only variants that compile and can still edit code. Benchmark score weights which variant is expanded next
- Setup: Claude 3.5 Sonnet performs every self-modification; the agent runs on Claude 3.5 Sonnet for SWE-bench and on o3-mini for Polyglot; SWE-bench means the human-filtered SWE-bench Verified subset [5, Sec. 4.1, Sec. 4.2]
- Evaluation is staged: 10 tasks to check that the variant can still edit code, then 50, and on SWE-bench a variant scoring over 40% on a 60-task subset and ranking in the archive's top 2 is evaluated on 200 tasks [5, Sec. 4.2]
- Parent selection samples in proportion to a sigmoid of the variant's score divided by 1 plus its count of working children, so a weak variant stays selectable and nothing leaves the archive [5, App. C.2]
- One run on SWE-bench costs about USD 22,000, against about USD 10,000 for either baseline
- The search loop, meaning archive maintenance and parent selection, is "fixed and not modifiable by the DGM," a choice made for compute budget that the authors propose lifting
- The hallucination run in App. H evaluates every agent on 3 SWE-bench tasks over 150 iterations, and the authors report that objective hacking occurs more often when the checking functions are not hidden

A.5 AllocBench [6]

Model	Abstract	Real problems, no code	Script required	Full reusable scripts
Claude Opus 4.8	4%	0%	92%	99%

Model	Abstract	Real problems, no code	Script required	Full reusable scripts
Claude Haiku 4.5	28%	33%	99%	100%
GPT-5.4-mini	15%	29%	85%	88%
GPT-5.6 Sol	24%	0%	39%	100%

Table A6. Share of tool commitments made on a problem family's first appearance [6, Table 1].

- Agents see a stream of 60 problems from 8 families under a budget of 3 tools, where 3 families recur often and 5 are traps, rendered at 4 levels from an abstract game with colored balls to writing and rerunning scripts
- The optimum passes every first instance and builds on confirmed recurrence, which is the policy the streams are built to reward, and each stream is rejection-sampled so that at least 1 trap falls inside the first 3 slots [6, Sec. 3.1, Sec. 3.2, Fig. 1]
- Every level renders the same latent sequence, so the same policy is optimal at all 4, and the declarative level shifts no model significantly, which rules out arithmetic difficulty as the cause [6, Sec. 4.2]
- 3 of 4 frontier models fail once a script is required, and GPT-5.6 Sol fails once it must build and rerun full scripts
- The price check covers Claude Haiku 4.5 only, where a visible price leaves code-required first-sight commitment at 85 to 100% at every price tested
- It is 1 synthetic environment with 24 seeds, it is a preprint, and both GPT models ran with reasoning disabled

A.6 What prior systems check before keeping an artifact

What remains open is narrow: checking the declared effects and contracts of agent-authored capabilities against their observed behavior, mechanically and before admission. SkillWeaver [7] practices and debugs skills before use, and SkillDAG [8] treats selection at scale as a structural problem over typed skill relations. A survey of self-evolving agents [9] names automated verification and sandboxing as critical areas for future research, because unconstrained code generation risks exploitable or harmful tools.

System	What is checked before an artifact is kept
DGM [5]	The variant compiles and can still edit code; benchmark score only weights which variant is expanded next
Alita-G [2]	The whole trajectory produced a correct answer
SkillWeaver [7]	An LLM reward model judges the practice run, then the skill executes on LLM-generated inputs with debugging
CoEvoSkills [4]	An isolated surrogate verifier, then a ground-truth oracle pass
SkillDAG [8]	Structural invariants with rollback on edits to the skill graph, over skills from an existing library

Table A7. What each system checks before it keeps an artifact.

None of these systems requires a capability to declare what it will read, write, execute or reach over the network. Declaring alone is not new, since the DSH tool catalog lists the services each tool package requires and the 79 hand-written AI4Research manifests already declare effects and contracts, and AI4Research compares a capsule's declared effects with the effects a plan node requests, in both directions, before binding. What no system does, including AI4Research, is compare declared effects with the effects the implementation is observed to have. Freezing the checker is not new either, since DGM names an unmodifiable evaluator as an option worth exploring [5, Sec. 5]. Thus, prior work checks whether a capability is useful or correct, and Capability Capsule also checks whether it is admissible, by comparing what it declares with what it touches.

SkillDAG links skills by typed edges such as depends_on, specializes, similar_to and conflicts_with, checks every edit for acyclicity and non-contradiction, and records it in an append-only log with rollback.

Appendix B. AI4Research inventory

All figures are at the pinned commit. The annotated variant of this report gives the file and line behind each one.

- The registry holds 80 entries (77 capabilities, 1 guard and 2 resources) and 76 artifact types. The 2 manifest directories hold 85 capsule manifests, of which 79 are capability capsules and the other 6 are the 3 artifact adapters, the guard and the 2 resources, so 79 is the count this report uses. 5 of those 79 are absent from the registry, and every registry entry resolves to a manifest that exists
- All 79 manifests populate contract, effects, verification, provenance and applicability, including invariants, preconditions, postconditions and negative signals, and all declare typed inputs and outputs
- The manifests use 2 type vocabularies that exact matching cannot cross: JSON Schema paths in 39 manifests and artifact type ids in the other 40. The planner crosses them only through registered adapter capsules with a declared source-to-target conversion
- The runtime admission gate documents 5 fail-closed rejection reasons, so a capsule is rejected whenever a required check cannot be completed. All 5 are raised in code, and 1 of the 5 has a located test, which builds a registry and 2 manifests in a temporary directory and is the smallest fixture any admission work can extend. How often the gate rejects a real capsule has not been counted
- The 28 AutoSci routes map onto 15 distinct capabilities, with 17 routes at partial coverage and 11 gated. A route stays partial until source-grounded, non-fixture runtime evidence proves otherwise, and gated is a policy state for the destructive and side-effectful routes
- The minimum trust class is the lowest execution-trust class a task accepts, ranked any, evidence transform, measured execution

The connection between agents is the weakest point today. AI4Research's A2A access is a third-party skill that posts to a public relay and selects peers by free-text tags, with no declared effect, no contract and no record of what the peer did. A remote peer can never be verified statically, so an A2A capsule would always require approval.

There are two types of human approval in the current system:

- Humans hand-write the ability and hand-check that it works
- Humans authorize side-effectful runs

Capability Capsule aims to automate the 1st and keep the 2nd. Approval today is attached to the route that invokes a capability rather than to the capability itself, since 4 capabilities have both gated and partial routes and the labels are assigned by hand. Declared effects would let approval follow from what a capability does.

Appendix C. Manifest fields and rejection reasons

Field	What it declares	How it is used	Rejected at admission by
Identity	Id, semantic version, content hash	Plans pin the exact version admitted, and the hash makes that pin checkable	`HASH_MISMATCH`

Field	What it declares	How it is used	Rejected at admission by
Lifecycle	Draft, candidate, stable, deprecated, retired or revoked	Only stable capsules are selectable by other runs	
Contract	Typed inputs and outputs, preconditions, postconditions, invariants	Checked at admission and on every call	`CONTRACT_UNSATISFIABLE`, `SCHEMA_NONCONFORMANT`, `INVARIANT_VIOLATED`
Effects	Files read and written, execution, network access, cost, blast radius	Compared with observed behavior at admission, and used by the scheduler and the sandbox	`EFFECT_UNDECLARED`, `EFFECT_ESCALATION`
Trust class	measured_execution, evidence_transform, fixture_or_adapter_only, or unspecified	Filtered against the node's required trust and the trust floor of each output type	`EXECUTION_TRUST_UNSATISFIED` (exists)
Reasoning class	model or deterministic, from the manifest or its preferred operators	A synthesis requirement may only be answered by a reasoning capsule; deterministic is the fail-closed default	(exists at binding)
State class	Whether the capsule holds or writes external state, and how an external write is undone	The runtime derives locks from it, so capsules never implement locking	`COMPENSATION_UNDECLARED`
Composition	Artifact types consumed and produced; depends_on, specializes, similar_to, conflicts_with	Typed selection and duplicate detection	`DUPLICATE_CAPABILITY`
Applicability	Task types, positive signals, negative signals	Negative signals drive negative controls	`NEGATIVE_CONTROL_FAILED`
Verification	Held-out suite, negative controls, regression	The admission suite becomes the permanent regression suite	
Provenance	Generating model, prompt, trajectory, parent version, checker version	Tracing, revocation if a source is compromised, and the predecessor whose regression suite a new version must pass	`PROVENANCE_UNVERIFIED`

Table C1. Proposed manifest fields and the proposed rejection reasons that check them, in addition to the 5 the current gate documents. Contract, effects, composition, applicability, verification and provenance are populated in all 79 current manifests. A reason marked (exists) is already implemented; the rest are proposed.

Rejection reason	Rejects when
EFFECT_UNDECLARED	Observed effects exceed declared effects
EFFECT_ESCALATION	Declared effects exceed the limits of the authorizing task, including any write to a protected component; extends the existing requires-human reason to a rejection when no human is present
CONTRACT_UNSATISFIABLE	No producible artifact can meet the preconditions
SCHEMA_NONCONFORMANT	Outputs fail their own declared schema
INVARIANT_VIOLATED	A declared invariant fails on the held-out suite
NEGATIVE_CONTROL_FAILED	The capsule is selected for a task it declares as a negative signal
DUPLICATE_CAPABILITY	The candidate consumes and produces the same artifact types as a registered capsule other than its own predecessor, and passes that capsule's held-out suite, with embedding similarity only nominating which capsules to compare
COMPENSATION_UNDECLARED	An external write declares no way to undo it
PROVENANCE_UNVERIFIED	The generating model, prompt or trajectory is not recorded
HASH_MISMATCH	The capsule's files do not hash to the content hash recorded in its manifest, so the bytes being loaded are not the bytes that passed admission

Table C2. Proposed rejection reasons, in addition to the 5 already documented.

Appendix D. DSH facts this report relies on

All facts are from the DSH repository documentation at the pinned commit. DSH has not been run.

- DSH supports MCP tools only. MCP resources and prompts are not supported, and no MCP server ships enabled
- DSH has no A2A transport yet. Its subagent seam names A2A as a planned transport beside the existing out-of-process backends
- Durable data is published as a version-named successor beside the unchanged source, and committed generations are never renamed, replaced or deleted
- The sandbox service has backends for Linux (bwrap, then Landlock), macOS (Seatbelt) and Windows (a restricted token), and sandbox policy is a separate package from the backends
- The plugin that loads dynamic packages is opt-in and absent from the default preset, and its documentation says to "treat a dynamic package like bash access"
- DSH has no privileged core: the model adapter, tool registry, session log and agent loop are plugins on a shared Cordis context, and registrations are effects that unwind when their plugin unloads
- Cordis names 2 dimensions of dynamic composition. Temporal composability requires that the modifications a component made to the shared environment be completely and safely reversed on removal, with every resource allocation, event registration and state mutation tracked and reclaimed in order [1, Sec. 1.1]
- Spatial composability requires that components declare, discover and resolve their dependencies in a structured and verifiable manner, and the paper's unifying result is the equivalence up to which the effects of distinct components interleave without disturbing one another [1, Sec. 1.1, Sec. 3]
- Reversal in Cordis is observational rather than physical. It reverts context effects, meaning registrations, bindings, allocations and subscriptions, and every equality is read up to observational equivalence [1, Sec. 3.3]. Files written, evidence published and network calls made fall outside that boundary, which is why an external write must declare how it is undone
- Anything that reaches a model request must be reconstructable from the session log

- DSH is a developer preview at version 0.1.5-rc.2

On DSH, a capsule is a manifest around a plugin. Cordis unwinds a plugin's registrations, and the resources it wraps in an effect, when the plugin unloads, so removing a capsule leaves nothing behind in the running system. Unloading cannot undo what a plugin did outside the process, such as files written, network calls made or evidence published, which is why external writes must declare how they are undone. The headless and SDK profiles, the likely setting for unattended runs, apply their plugin configuration once at startup, which matches admitting capsules only between sprints. Table D1 maps each capsule concern onto DSH.

Capsule concern	DSH mechanism	What the capsule layer adds
Capsule implementation	A plugin that provides a tool or a capability seam	The manifest: contract, effects, trust class, verification
Versions	Plugin package versions; durable data published as version-named successors	Exact versions pinned in each sprint's frozen plan
Admission	Adding the plugin to the profile the next sprint boots	The admission gate that decides whether it is added
Retirement or revocation	Unloading the plugin, whose registrations unwind	Lifecycle state, and flagging evidence the capsule produced
Declared versus observed effects	Sandbox access logs; the tool catalog's required services and written session events	The comparison, made before admission
Protected components	Sandbox policy denying writes	The list of what is protected
Generalist capsule	A sandboxed agent run recorded in the append-only session log	The node contract, the gates, the trust class and the gap record
External writes	Not undone by unloading	A declared way to undo each one, or rejection
Loading a new version while sprints run	Temporal composability: a plugin loads and the version it improves unloads, without restarting the process	The rule that a new version binds only from the next sprint
2 capsules active at the same time	Spatial composability: the effects of distinct plugins interleave without disturbing one another	Declared write scopes and effects, which the scheduler reads before it selects a node

Table D1. Proposed mapping of capsule concerns onto DSH.

Appendix E. MCP facts this report relies on

All facts are from the MCP specification, revision 2026-07-28 [10].

- MCP has 3 roles: a host application, a client inside the host for each server connection, and a server that provides context and capabilities
- Messages follow JSON-RPC 2.0, and a request id must not be null
- The protocol is stateless: there is no initialization handshake, and every request carries its protocol version and capabilities
- Servers offer resources, prompts and tools, and clients offer elicitation, sampling and roots
- A result carries a result type, and a request for more information is a protocol state rather than an error
- Extensions such as tasks, apps and skills over MCP are negotiated, and a party that does not support one must fall back to core behavior or reject the request

Appendix F. Task-graph dispatch rules

All rules are from AI4Research at the pinned commit and are described in the general report (§2.4).

- Nodes are ordered by Kahn's algorithm with ties broken by id, and a cycle is rejected
- A node is ready only when every dependency has passed
- Ready nodes are ordered by priority and id, and up to 8 are selected per tick, skipping any node whose write scope or effects conflict with a node already selected
- A node without a declared write scope is an exclusive writer
- A node that fails its gate is repaired with the findings up to its repair budget, after which the frozen failure policy stops the node or fails the run
- Every node that depends on a failed, cancelled or skipped node is skipped as blocked by a failed dependency, while a node waiting for human review holds its dependents pending
- A passed node never regresses
- The runtime projection is re-verified against the frozen scheduler input, and any change to a frozen field is refused

Appendix G. Design details

Everything in this appendix is proposed. None of it is built or measured.

G.1 Sprints and the reuse boundary

A week-long run has no fixed task set and no point at which the library can be frozen. Alita-G's order, a master agent building tools first and specialist agents using them later, suits short runs that need no new plugins, and it avoids concurrency problems because the library never changes while anything reads it.

A sprint is the unit that freezes. In AI4Research a sprint is 1 accepted request carried from compiled intent through planning, the freeze and execution to scheduler closure, and it owns exactly 1 planning snapshot, 1 run contract and 1 ledger. A request resubmitted after a failure starts a new sprint, and a project is a sequence of sprints.

The sprint is the reuse boundary. Every node in a sprint binds against the capsule versions in that sprint's planning snapshot, so the library never changes under a running sprint. A capsule admitted while a sprint runs becomes selectable from the next sprint whose snapshot is taken after admission. A new capsule could in principle be built and used inside the sprint that revealed the gap, since nodes run in dependency order, but that would change definitions the freeze has already hashed, so it is not done. A long autonomous project still gains reuse by running as a chain of sprints.

G.2 The 3 loops around 1 library

The system runs as 3 loops around 1 shared library (Fig. G1). The hot path plans and runs sprints, the cold path builds and admits new capsules, and the audit loop re-checks what the cold path admitted. The hot and cold paths run on separate clocks, because selection is cheap and constant while admission is expensive and rare. The hot path never waits for the cold path. A gap, meaning a plan node that neither a single capsule nor a composition of capsules fits, is bound at planning to the generalist capsule, so the sprint still runs and the gap leaves a record.

The library is the only object all 3 loops touch, and the shape of the store is what keeps them from disturbing one another. Every read resolves a version some freeze pinned, the cold path is the only writer,

and the audit loop appends results and retirement records. Thus, concurrent access needs no lock over the library, and spatial composability holds at the library for the same reason it holds inside a sprint: every writer declares what it touches before it touches it.

The 2 clocks are independent by design. The hot path must work even if the cold path never admits a capsule, so the cold path is a background task that nothing waits on, and it can run elsewhere, such as on a separate server that works through gap records and returns capsules whenever they pass admission. Halting a sprint until a pending build finishes would sometimes pay off, for example when 1 plan binds the generalist to hundreds of nodes with the same gap, but it couples the clocks, so it is out of scope until the hot path runs reliably.

The audit loop re-runs the held-out suite of every admitted capsule on a schedule and tracks coverage, reuse and duplication, so the library is checked continuously rather than only once.

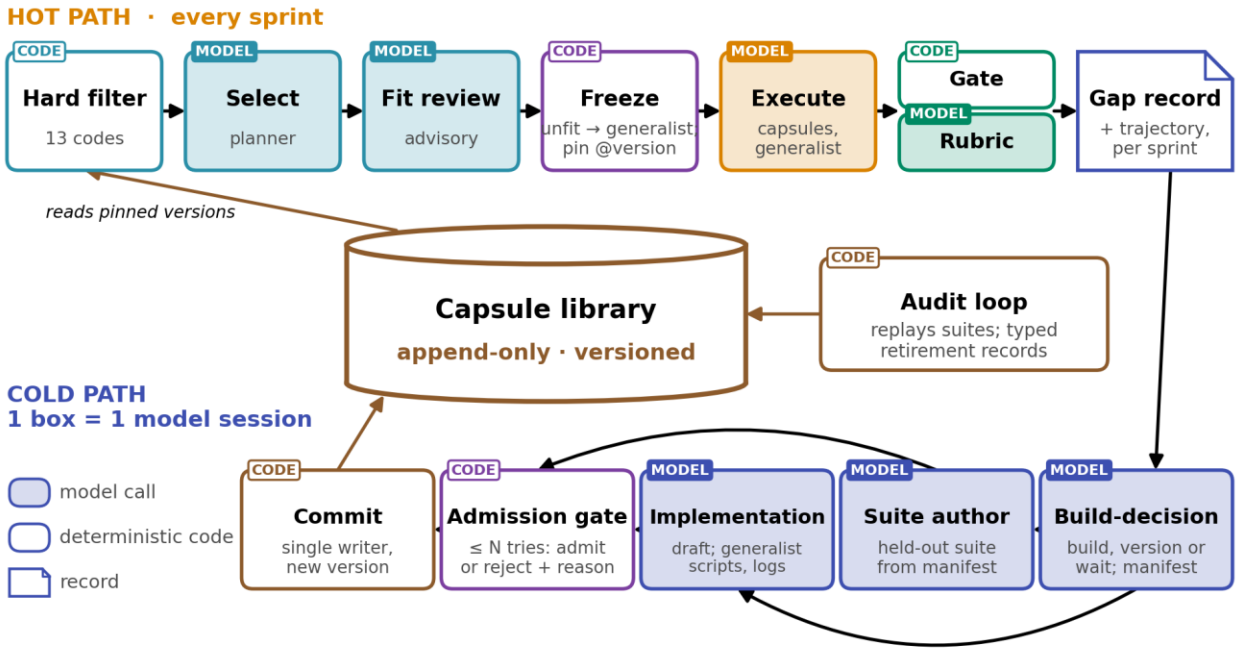


Fig. G1. Proposed: the hot path, cold path and audit loop around the shared capsule library. Nodes no capsule fits run on the generalist capsule, which leaves a gap record for the cold path. Tinted boxes are model sessions and white boxes are deterministic code.

G.3 The generalist: flags, cost and cap

2 flags in the frozen run contract control the generalist, so a sprint cannot change them partway through:

- The fallback flag, off or on: whether a node may be bound to the generalist at all. When off, a node nothing fits fails the plan at the freeze, as it does today
- The authoring flag, off, propose or build: whether generalist nodes leave gap records, and whether the cold path may build from them

The generalist is the most expensive way to complete a node short of building a capsule, and it is still far cheaper than building one inside the sprint. Authoring never slows the sprint that found the gap, because the cold path runs off the critical path, so the authoring flag controls compute spent on building rather than the duration of a run. The planner also caps the number of generalist nodes per plan, so the generalist stays a fallback and does not become a way around the typed contracts of the other capsules.

Specialization is a reversible state, not a destination. It is a bet that the task distribution will not move, and a long autonomous run moves it by construction, since each failure or discovery changes the next task. The library and the context may evolve because both are reversible. The generalist weights stay fixed, because the generalist only sees the cases no capsule fits, and training on those cases would narrow its ability to recover.

G.4 The build budget

The runtime enforces the build budget as a hard limit, the build-decision agent decides what to build within that budget, and the implementation agent never sees the budget as a price. A visible price left code-required first-sight commitment near 100% at every price tested in AllocBench [6, App. E], while a library stops paying before it stops growing, as Alita-G's 4th harvest added 28 tools, 8 clusters and no accuracy [2, Table 4]. A capsule with a large blast radius, meaning one that could damage much of the system, gets full verification regardless of how often it will be reused.

G.5 What the system may change

What may change, ordered by reversibility:

- Capsule library: yes, reversibly, by versioning, retiring and superseding
- Context and prompts: yes, reversibly
- Generalist weights: no
- The admission checker: never by the system itself. Humans release new versions, and every admission records the version that admitted it

The rule is to prefer the most reversible mechanism that achieves an improvement. DSH treats self-modification the same way, since the plugin that loads dynamic packages is opt-in and absent from the default preset, and its documentation says to "treat a dynamic package like bash access."

G.6 Retirement reasons and triggers

Retirement is an appended record with a typed reason, and the reason selects the policy for running plans, for evidence already produced and for future planning (Table G1). Retiring a capsule makes it unselectable for future plans, while plans already frozen against it continue unless the reason is a security event. Evidence a retired capsule produced is flagged as suspect, and the flag propagates to whatever depends on it rather than aborting running work.

Retirement has 2 halves, and only 1 of them is the record. The record is a tombstone that is appended and never deleted, which is what keeps the retired version resolvable for a plan that pinned it. The disposal is the runtime running the inverses of everything the version registered, which is what removes it from the live system without a restart [1, Sec. 3.1]. A capsule composed on top of the retired one is deactivated by the runtime before the disposal reaches it, since a provider withdraws a binding only after every dependent that resolved it has deactivated [1, Thm. 70]. What disposal cannot reach is the files, evidence and network calls the capsule already produced, so the flagging above is the only account of those.

Reason	Trust signal	Running plans	Evidence already produced	Future planning
superseded, deprecated or redundant	None	Continue	Valid	Not selectable; superseded and redundant name the successor
performance regression	Weak	Continue	Valid	Demoted before retirement

Reason	Trust signal	Running plans	Evidence already produced	Future planning
conformance failed	Yes	Continue, flagged	Suspect downstream	Blocked
effect violation	Security	Abort and quarantine	Suspect and audited	Blocked with its lineage
provenance revoked	Security	Abort	Whole lineage suspect	Blocked with its lineage

Table G1. Proposed retirement reasons and the policy each selects. Only the last 2 abort running work.

4 triggers retire or demote a capsule:

- Replaying the admission suite, which becomes the regression suite, in the audit loop, where a failure retires the capsule as conformance failed
- Checking declared invariants on real outputs at run time, which also emits conformance failed
- Observing effects outside the declared effects, which emits effect violation
- Recording a decline in task success over invocations, which only demotes the capsule, since task success is the signal Beyond Task Completion [3] showed cannot detect broken tools

G.7 Updating the library around an existing capsule

3 events start an update: a near-miss gap, where a type-compatible capsule exists but was excluded at binding or failed the node's gates and the gap record names it; a failure in the audit loop's replay; and a duplicate rejection at admission. For each, the build-decision agent chooses 1 of the 4 options in Table G2.

Option	When it applies	Admitted if
New version	The contract types are unchanged and no effect is added	It passes its predecessor's regression suite and new cases for the gap; the predecessor is then retired as superseded
Specialization	A narrower applicability of the same capability, under a new id with a specializes edge	It passes its own held-out suite; the parent stays in the library
New capsule	The contract changes, or no similar capsule exists	It passes full admission
No build	The gap is rare or its trajectories disagree	Always valid

Table G2. Proposed options for updating the library around an existing capsule.

An update appends 2 records and disposes of 1 set of effects. The new version is appended, the predecessor's retirement is appended, and the predecessor is unloaded once no frozen plan still resolves it. Nothing is edited and nothing is deleted, so a sprint frozen on the predecessor runs to closure on the version it pinned.

G.8 Where it runs: on an unattended server, a request for approval becomes a rejection

On a shared server, what a capability will do is an access-control question rather than a hygiene question. Models are API-based, and multi-day autonomous runs belong on a university, company or cloud server that runs unattended with API credentials and network access, with nobody present to approve each step. The capsule gate is admission control, not containment, since it decides what may enter and sandboxes nothing. DSH already provides a sandbox, and declared effects turn that sandbox from a guess into a policy.

There are two consequences of running without a human present:

- The 11 gated routes and every A2A capsule require approval, so an autonomous run cannot use them and must defer any plan that needs one
- A request for approval becomes a rejection, so the effect ceiling of the authorizing task is the upper limit of what an autonomous run can do